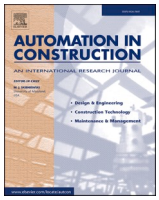
Automation in Construction 183 (2026) 106805



Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon





Self-hosted multimodal large language models for speech-driven perception and navigation in construction robotics

Muhammad Anas Gopee [a,*], Samuel A. Prieto [b], Borja García de Soto [a]

[a] S.M.A.R.T. Construction Research Group, New York University Abu Dhabi (NYUAD), United Arab Emirates
[b] KINESIS Core Technology Platform, New York University Abu Dhabi (NYUAD), United Arab Emirates

ARTICLE INFO

Dataset link: Repository of the framework, source code, and prompts used in this study

Keywords:
Human–robot interaction
Function calling
Edge deployment
Data locality
Privacy-preserving control
Hybrid architecture

ABSTRACT

This paper explores self-hosted, open-weight multimodal large language models (LLMs) as a speech–vision interface for robot perception and navigation. Using structured function calling, the approach enables transparent control without application-specific training. Two data-local deployment modes are compared: a smaller on-robot model and a larger edge-server model. Both convert spoken commands and images into structured function calls for image capture, analysis, and waypoint navigation. Laboratory evaluations show that the edge-server model outperforms the on-robot model in speed and reliability, particularly in visual tasks, while the on-robot model performs competitively in simple workflows. Findings support hybrid deployment strategies that combine edge-hosted and on-robot models to balance performance and resilience, enabling responsive, cloud-independent robotic operation in construction environments.

1. Introduction

The construction sector has lagged other industries in automation due to the complex and dynamic nature of job sites. Environments are cluttered, tasks change frequently, and the workforce is diverse, often without specialized training in robotics. Conventional human-robot interaction (HRI) tools, such as programming interfaces or tablet-based controls, are unintuitive for untrained crews ([1]). These usability barriers are compounded by infrastructure constraints unique to construction: partially enclosed structures limit wireless connectivity, dusty and noisy conditions challenge sensing systems, and the presence of workers in close proximity to equipment demands robust safety mechanisms [2]. At the same time, cloud-dependent robotic systems suffer from variable latency, unreliable connectivity in remote or network-limited sites, and concerns over data privacy, all of which limit their practicality for safety-critical, real-time operations [3].

Against this backdrop, robotics for construction has shown promising results in specific tasks such as inspection, logistics, and basic manipulation, yet adoption remains constrained by usability and integration barriers [4]. Systems that rely on extensive, application-specific datasets are slow to adapt to changing site conditions and require significant investment in data collection and model training. Interfaces that presume technical expertise create friction for front-line workers who need to interact with robots as part of their daily workflows, not as specialized operators. The combination of diverse crews, noisy environments, and safety-critical operations underscores the need for robust, accessible, and context-aware communication channels that can function reliably under real-world construction constraints.

Recent progress in large language models (LLMs) and multimodal artificial intelligence presents an opportunity to overcome these barriers. LLMs can interpret user intent from natural language, reason over both textual and visual information, and produce structured outputs that map to robotic control functions, enabling accessible and bidirectional communication without requiring users to learn specialized command syntax [5]. The ability to process speech and images through a unified interface is particularly valuable in construction contexts where workers' hands may be occupied, visual confirmation is essential for safety, and diverse language backgrounds necessitate flexible interaction modalities [6].

However, a fundamental methodological choice emerges when applying LLMs to safety-critical construction robotics: end-to-end vision-language-action (VLA) models versus structured function calling approaches. VLA models offer the appeal of unified learning across perception, language, and action, potentially reducing engineering

* Corresponding author.
E-mail addresses: nas@nyu.edu (M.A. Gopee), samuel.prieto@nyu.edu (S.A. Prieto), garcia.de.soto@nyu.edu (B. García de Soto).

https://doi.org/10.1016/j.autcon.2026.106805
Received 17 November 2025; Received in revised form 23 January 2026; Accepted 26 January 2026
Available online 31 January 2026
0926-5805/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

effort [7]. Yet they produce black-box behaviors with opaque input-action mappings, making it difficult to verify correctness, diagnose failures, or provide safety guarantees, critical requirements in construction where equipment malfunctions can endanger workers. VLAs also typically require substantial demonstration data for training and are often computationally intensive, limiting their deployability on resource-constrained platforms. In contrast, structured function calling positions the LLM as a reasoning engine that generates explicit, verifiable commands. Each function call can be logged, validated against safety constraints before execution, and mapped to established robotic middleware such as ROS [8].

This transparency enables human supervisors to audit decision chains, developers to modularly extend capabilities, and safety systems to intercede when necessary. For construction applications where accountability, interpretability, and integration with existing infrastructure are paramount, structured approaches offer practical advantages despite requiring more deliberate system design. This can also reduce the barrier to adoption given the HRI functionality can be added as a layer on top of existing workflows rather than requiring a complete system overhaul as would be the case with VLA architectures.

Building on this structured paradigm, this paper presents a self-hosted multimodal framework for speech-driven perception and navigation that translates natural language requests into verifiable robot behaviors while maintaining complete data locality. The framework integrates speech recognition, a vision-capable language model with structured output generation, and speech synthesis to enable intuitive, hands-free interaction with visual context awareness. Running these components on-site eliminates cloud dependencies, ensures predictable latency, and addresses privacy concerns, particularly important in construction where site documentation and worker presence are sensitive.

A central challenge in deploying LLM-based interfaces for construction robotics lies in balancing model capability against computational and infrastructure constraints. Larger models generally offer superior reasoning and reliability but require more powerful computing resources, while smaller models can run on embedded platforms but may sacrifice performance. To investigate these tradeoffs, two self-hosted deployment modes that preserve data locality were compared: a smaller model running entirely on the robot's embedded compute, and a larger model hosted on a co-located edge server within the local network. This comparative design enables the characterization of when on-robot deployment suffices versus when local edge infrastructure provides worthwhile advantages, informing practical deployment decisions for construction sites with varying computational resources and connectivity profiles. Both configurations were evaluated across construction-relevant workflows of increasing complexity, including safety monitoring scenarios, focusing on responsiveness, functional reliability, and the ability to sustain natural dialogue without task-specific fine-tuning. Few-shot safety guidance capabilities were also explored, achievable through prompt engineering alone, examining whether useful behaviors can emerge without application-specific training or demonstration data collection.

The contributions of this work are: (i) a self-hosted multimodal framework employing structured function calling for verifiable, transparent robotic control tailored to construction HRI while preserving data locality; (ii) a systematic empirical comparison of on-robot versus co-located edge deployment modes, characterizing performance-resilience tradeoffs across model scales and task complexities to inform practical deployment decisions; (iii) task-specific analysis of perception and navigation capabilities that reveals which functions are most sensitive to model capacity, providing guidance for hybrid architectures; and (iv) demonstration that few-shot operation without application-specific training or demonstration data can achieve functional performance for construction-relevant tasks, reducing integration barriers and training overhead.

The remainder of this paper is organized as follows: Section 2 reviews related work in LLM-enabled robotics, edge deployment strategies, control paradigms, and navigation systems; Section 3 describes the framework architecture and deployment modes; Section 4 details the experimental protocol and robotic platform; Section 5 presents quantitative results and statistical analysis; Section 6 acknowledges limitations; and Section 7 presents conclusions and future directions.

2. Background and related work

Robotic systems in construction face persistent challenges in usability, reliability, and integration. Traditional approaches rely on cloud-based processing, application-specific training, and complex interfaces, limiting practicality in dynamic, bandwidth-constrained sites. This section reviews advances in LLM-enabled human-robot interaction, edge deployment strategies, control paradigms, and navigation systems, highlighting gaps that motivate self-hosted, speech-driven robotic interfaces for construction.

The work of Gopee et al. [3] reviewed on-device LLMs for construction robotics across two areas. First, they examined LLMs for robotics, noting that while surveys demonstrated their potential for adaptable reasoning and HMI, most implementations relied on cloud servers with associated latency, connectivity, and privacy concerns. They also discussed edge deployment techniques (quantization, pruning, optimized inference) and found that while quantization-aware training reduces accuracy loss, early edge deployments struggled with command misinterpretation. They identified distilled models on off-the-shelf edge platforms as more promising than costly custom hardware solutions.

A cloud-based multimodal LLM interface for construction HRI was also presented in ([1]). This work emphasized construction's dynamic, cluttered, and hazardous nature compared to manufacturing, with system rigidity and HRI complexity limiting adoption. The review covered intuitive interfaces and multimodal approaches, while identifying gaps in multilingual support, real-time decision-making, and real-world evaluation, gaps persisting in cloud-dependent architectures.

Building on these foundations, this review expands coverage in four areas: (i) HRI using LLMs in Construction, (ii) edge and self-hosted LLM deployment for robotics, (iii) Vision-Language-Action Models and structured control approaches, and (iv) LLM-driven robot navigation, addressing the evolution toward practical self-hosted solutions.

2.1. Human-robot interaction with large language models in construction

Yu et al. [9] address fundamental conceptual gaps in construction robotics through systematic review of HRI and HRC applications. Analyzing 117 papers, they define HRI as humans exchanging information with robots through interfaces for control/communication, and HRC as human-robot collaboration toward shared goals. Critically, they identify that HRI serves HRC by enabling communication, while HRC without interaction becomes rigid in dynamic construction environments. They propose Human-Robot Partnership (HRP) as the integration of both, with key elements including interface design and information recognition (HRI), task allocation and robot capability (HRC), and human well-being spanning both, establishing that effective construction robotics requires seamless integration of interaction and collaboration.

Kim et al. [10] demonstrated practical LLM implementation using ChatGPT-4 as the cognitive core for autonomous construction robots, integrating BIM data with natural language dialogue for task planning. Their mobile painting robot adaptively executes tasks through real-time LLM conversations processing structured BIM and unstructured human inputs, reducing reliance on hardcoded logic while enabling flexible task planning.

Park et al. [6] expanded multimodal capabilities with a conversational virtual reality (VR) interface integrating voice and controller inputs with ROS, BIM, and LLM-powered chat. Evaluated with 12

construction workers on drywall installation, the system demonstrated low workload and high intuitiveness, advancing beyond single-modality approaches by combining speech with spatial gestures. Yang et al. [11] addressed worker well-being through a multi-agent conversational AI using LLMs and RAG, integrating construction knowledge with specialized agents simulating group conversation. With 12 participants, the system significantly improved usability, self-determination, social presence, and trust over single-agent baselines.

While these implementations show promise, they rely predominantly on cloud-based models (ChatGPT/GPT-4), inheriting latency, connectivity, and privacy limitations. Wang et al. [12] survey multimodal HRI in manufacturing using a perception-cognition-action framework, yet these approaches have not been adapted to construction's constraints: noise, dynamic environments, and safety-critical operation in network-limited settings.

2.2. Edge and self-hosted LLM deployment for robotics

Cloud dependency introduces critical challenges: variable latency, unreliable connectivity, and privacy concerns. Recent advances in knowledge distillation and optimization have produced compact models for edge deployment in robotics.

Ravichandran et al. [13] introduced PRISM, distilling small language models (SLMs) from teacher models for robot planning with minimal supervision. PRISM synthesizes diverse tasks/environments, elicits teacher plans, and distills compact SLMs as drop-in replacements. Their Llama-3.2-3B achieves over 93% of GPT-4o's performance across mapping, exploration, manipulation, and assistance tasks, demonstrating that distillation preserves reasoning while enabling deployment across heterogeneous platforms.

Chen et al. [14] developed FASTNav using fine-tuning and teacher-student iteration for multi-point navigation, enabling local edge deployment with low cost, high accuracy, and low latency. Elksnis et al. [15] compared quantized local LLMs (Gemma, Llama3, Mistral) for semantic navigation on laptops, finding Llama3 best for room classification and goal selection. Nasrat et al. [16] demonstrate that fine-tuned models operate on devices with 8GB memory, integrating visual perception and speech recognition for on-device decision-making with enhanced contextual awareness.

Qu et al. [17] survey mobile edge intelligence for LLMs, covering caching, training, inference, and 6G integration, emphasizing privacy and latency benefits for delay-sensitive applications. Cai et al. [18] propose Edge-LLM, a collaborative framework distributing LLM inference and fine-tuning across edge infrastructure, addressing resource allocation and load balancing.

Hybrid deployment architectures balance performance and constraints. Zhu et al. [19] optimize edge inference allocation across distributed nodes, trading off latency, bandwidth, and quality. Baumann et al. [20] demonstrate complete on-board LLM deployment for autonomous driving, eliminating cloud dependency via local deployment on Jetson Orin AGX for safety-critical real-time decision-making.

Despite progress, gaps remain for construction applications. Most work evaluates single-task benchmarks rather than multimodal, multi-task scenarios integrating perception, navigation, and manipulation. Hybrid architectures lack systematic evaluation of performance trade-offs across network conditions, task complexities, and deployment modes. Construction-specific constraints, such as battery platforms, harsh environments, and safety-critical operation, have also received limited attention.

2.3. Vision-language-action models and structured control approaches

Kawaharazuka et al. [7] survey Vision-Language-Action (VLA) models for robotics. VLAs unify vision, language, and action data to learn policies generalizing across tasks, objects, embodiments, and environments with minimal task-specific data. The survey covers VLA strategies, architectures, learning paradigms, platforms, data collection, and benchmarks, revealing that VLAs offer end-to-end sensorimotor control by jointly learning perception, language understanding, and action generation.

However, VLAs face limitations for safety-critical construction. Their end-to-end nature produces black-box behaviors with opaque input-action mappings, making it difficult to verify correctness, diagnose failures, or provide safety guarantees. Fine-tuning requires substantial demonstration data, and models are often proprietary or computationally prohibitive for edge deployment. VLAs also struggle with explicit reasoning over structured constraints, multi-step planning, and integration with existing middleware, which are critical for construction.

In contrast, structured function calling positions the LLM as a reasoning engine generating explicit, verifiable commands. Manduzio et al. [8] demonstrate that small LLMs can be trained for reliable function calling using RLHF and DPO. Their framework queries LLMs with function descriptions, managing calls in reasoning chains. Building upon this idea, Prieto et al. [21] introduced a structured multi-agent framework that applies the principles of function calling to construction robotics. Their system employs two collaborative LLM agents, one acting as a Planner to generate structured task instructions in JSON schema, and another as a supervisor to validate these instructions before execution. By structuring all communications as verifiable API-like calls between agents and robot controllers, their approach demonstrates how function calling can be operationalized for embodied systems in safety-critical environments.

Gong et al. [22] extend this with GhostShell, enabling concurrent embodied programming through streaming function calls. By decomposing capabilities into composable functions and maintaining dialogue context, GhostShell enables parallel task execution. Zhang and Ji [23] apply modular function calling to navigation, positioning the LLM as central controller with dynamically composed tools, demonstrating context-aware navigation through transferable prompt templates.

Structured function calling offers key advantages for construction-related applications: verifiability (explicit calls logged and validated before execution), modularity (capabilities developed/tested independently), interpretability (transparent reasoning chains), integration (natural mapping to ROS infrastructure), and edge compatibility (smaller models run on constrained devices).

While VLAs represent important research, structured approaches currently provide a more practical path for safety-critical, resource-constrained construction where transparency, modularity, and established infrastructure integration are paramount.

2.4. LLM-driven robot navigation

Recent developments demonstrate LLMs' capacity to transform robot navigation through natural language understanding and semantic reasoning. Traditional navigation approaches relying on pre-built maps or reinforcement learning struggle with generalization, dynamic adaptation, and human-like command interpretation. LLM-based alternatives leverage semantic comprehension and zero-shot reasoning to enable more flexible, context-aware path planning aligned with natural human instructions.

Zhang and Ji [23] exemplify the modular approach, positioning the LLM as central controller with function calling capabilities. Their framework decomposes conventional navigation functions, such as localization, path planning, and obstacle avoidance, into reusable LLM tools with expandable configurations. Using systematically designed, transferable prompt templates, the framework adapts across implementations and environments, demonstrating context-aware navigation through dynamic tool composition in PyBullet simulation. This modular

architecture enables the LLM to reason about navigation goals while delegating execution to specialized components.

The integration of LLMs with edge deployment has become increasingly practical. Elksnis et al. [15] systematically compare quantized local LLMs (e.g., Gemma, Llama3, Mistral) for semantic navigation on consumer-grade laptops, evaluating both room classification and goal selection tasks. They find Llama3 achieves optimal performance for semantic navigation without cloud dependency, demonstrating that local LLMs can handle navigation reasoning on modest hardware. Chen et al. [14] advance this further with FASTNav, employing fine-tuning and teacher-student iteration to create adaptive small language models specifically optimized for multi-point navigation. Their approach achieves local edge deployment with low cost, high accuracy, and minimal latency, validated in both simulation and physical robots.

Extensions to probabilistic and multi-agent paradigms broaden LLM navigation capabilities. Latif [24] introduces 3P-LLM, incorporating probabilistic reasoning to handle uncertainty, enabling path planning that accounts for dynamic obstacles and incomplete environmental information. Samarathunga et al. [5] extend multi-agent concepts to navigation, employing LLM-guided collaboration among multiple agents to handle complex natural language-based scenarios requiring coordination.

Speech-driven control represents a natural interface for LLM navigation systems. Gupta et al. [25] demonstrate speech recognition-based wireless control for mobile robots, establishing foundations for voice-activated navigation. However, their approach employs simpler speech-to-command mapping rather than the contextual reasoning and semantic understanding that LLMs enable, representing an earlier paradigm that lacks the flexibility and adaptability of LLM-driven natural language interfaces.

Despite these advances, several critical gaps persist. Most LLM navigation research evaluates systems in controlled laboratory settings or structured environments with reliable sensing infrastructure. The integration of speech-driven natural language input with LLM reasoning for waypoint navigation in self-hosted configurations, eliminating cloud dependencies while maintaining data locality, has not been systematically demonstrated. Furthermore, structured function calling approaches that generate verifiable navigation commands for transparency and safety have received limited attention in LLM navigation literature, particularly in applications requiring safety-critical operation.

2.5. Research gaps and contributions

The literature reveals critical gaps at the intersection of self-hosted LLM deployment, multimodal interaction, and construction robotics. While LLM-enabled HRI has been demonstrated in construction contexts, implementations rely predominantly on cloud-based models (ChatGPT, GPT-4), introducing latency, connectivity dependencies, and privacy concerns inappropriate for bandwidth-constrained construction sites. Self-hosted multimodal systems that eliminate cloud dependency while supporting speech, vision, and structured control remain largely absent from construction robotics research.

Speech interfaces and LLM-based navigation have advanced independently, yet their integration for speech-driven waypoint navigation in self-hosted configurations maintaining complete data locality has not been systematically investigated. Existing speech-driven systems employ cloud models or simple command mapping, while local LLM navigation research focuses on text-based interfaces, leaving a gap in practical, voice-activated, self-hosted navigation for construction applications.

Although hybrid deployment architectures (on-robot vs. co-located edge server) have been proposed conceptually for LLM robotics, systematic empirical evaluation characterizing performance trade-offs across varying deployment modes, model scales, and task complexities remains limited. Understanding when on-robot deployment suffices versus when co-located edge infrastructure provides advantages is critical for practical construction robotics deployment decisions.

Finally, while structured function calling has emerged as an alternative to end-to-end VLA approaches, its application for zero-shot safety guidance in construction through prompt engineering alone and without application-specific training or demonstration data, has not been thoroughly explored, despite its importance for rapid deployment in safety-critical environments.

These gaps are addressed by presenting a self-hosted multimodal framework integrating speech-driven perception and waypoint navigation, employing structured function calling for verifiable control, systematically evaluating on-robot versus co-located edge server deployment modes, and demonstrating few-shot safety guidance capabilities through prompt design, advancing practical LLM-enabled construction robotics prioritizing data locality, verifiable operation, and deployment flexibility.

3. Methodology

This section describes the self-hosted multimodal framework for speech-driven perception and navigation. The goal is to enable natural, bidirectional communication between workers and mobile robots while keeping data on site and minimizing operator training. The framework couples push-to-talk voice interaction, a vision-capable language model with structured outputs, and a command layer that maps model intents to robot functions such as image capture and waypoint navigation. Fig. 1 shows an overview of the methodology process.

3.1. User interaction and multimodal input

Interaction is initiated by a push-to-talk trigger to ensure robust operation in noisy environments and to bound audio capture without the need of a wake-up word. While the trigger is pressed, speech is recorded and transcribed locally by an Automatic Speech Recognition (ASR) module. The transcript is provided to the multimodal language model, which may also request visual context when needed. Responses are delivered through a local Text-to-Speech (TTS) engine so that users receive immediate, spoken feedback.

3.2. Multimodal model and decision-making

The language model is guided by an application-specific system prompt that: (i) defines the assistant's role and communication style, (ii) enumerates the high-level functions it can call, and (iii) establishes safety and clarification rules. The model produces deterministic, structured outputs suitable for parsing (e.g., JSON-like fields) and is configured to request visual input when required. Typical functions include capturing an image for scene understanding, performing a safety check over the latest image, and initiating movement to a named waypoint. Dialogue context is retained across turns to avoid repetitive user instructions and to support follow-up clarification when requests are ambiguous.

3.3. Structured function calling and guardrails

Function calling serves as the bridge from language to action. Each function is specified by a name and a bounded set of arguments (e.g., a waypoint identifier or image parameters). The system verifies function names and validates arguments before execution. If outputs are malformed or incomplete, the assistant falls back to dialogue for clarification rather than executing a risky or ambiguous action. For actions with safety implications (e.g., movement), optional confirmation prompts can be required before dispatch.

Below is an example JSON structure that the LLM generates when calling functions:

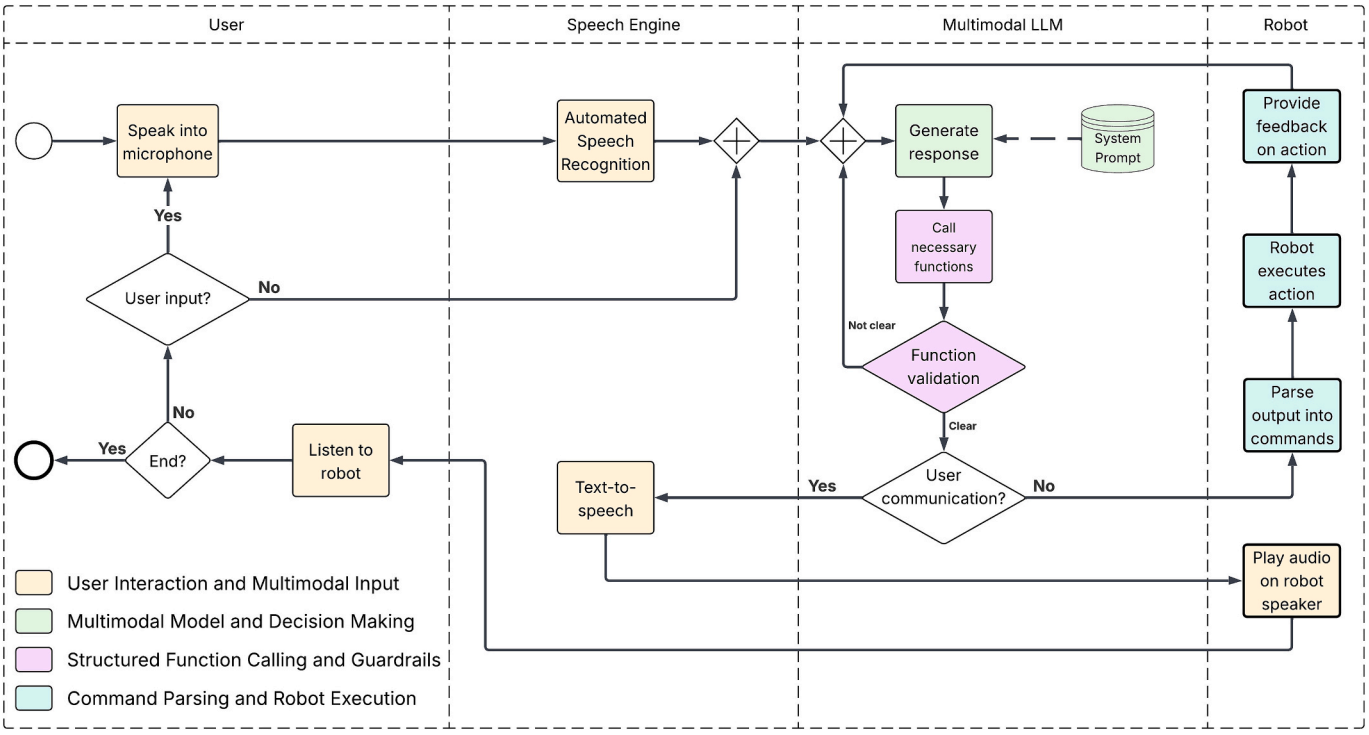


Fig. 1. Overview of the methodology with key phases and elements.

```json
{
"function": "name_of_function",
"params": {"param_1": "value_1"},
"speak": "Text to be spoken"
}
```

The structured function-calling architecture enforces several guarantees of robustness by design. A key property is formalized below.

Lemma 1 (Bounded Action Space): *The set of executable robot actions A is finite and bounded by the predefined function schema F = {navigate_to, take_picture, analyze_image, speak, activate_safety_protocol}. For any LLM output o, the executed action a satisfies a in A or a = null (no execution).*

Proof sketch: The system validates each LLM-generated JSON output against the fixed schema F before execution. The validation step (i) checks that the function field matches an element of F, (ii) verifies that required parameters are present and conform to type/constraint checks, and (iii) rejects malformed or incomplete outputs. For example, navigate_to only accepts predefined waypoint identifiers (not arbitrary coordinates), speak enforces a bounded text field, and activate_safety_protocol takes no free-form parameters. If validation fails, no action is dispatched; instead, the system falls back to dialogue for clarification. Since F is finite (five functions) and parameter domains are bounded, the robot can only execute actions within this constrained set. Evidence: see the Example JSON structure for function calling, the functions available to the LLM (Section 4.1.7), and the validation-before-execution description in Section 3.4.

3.4. Command parsing and robot execution

Validated function calls are mapped to robot capabilities by a command parser. Image requests trigger the robot camera pipeline and return a reference to the captured frame for downstream reasoning. Navigation requests invoke the robot's navigation stack to move toward predefined waypoints or provided poses, reporting status updates (e.g., en route, reached, or aborted). Execution feedback is streamed back into the dialogue context, enabling the assistant to confirm outcomes, recover from failures, or request user input when obstacles or uncertainties arise. Fig. 2 shows an illustration of the communication between the LLM and ROS2 for navigation tasks using the python script as middleware.

3.5. Deployment modes (on-robot and co-located edge server)

To preserve privacy and reduce network dependencies, all core components are hosted on site using two practical modes:

- Fully on-robot: ASR, the language model, TTS, and the command parser run on the robot's embedded compute. This mode provides predictable, offline operation with minimal infrastructure and consistent latency.
- Co-located edge server: The robot hosts ASR, TTS, and the command parser, while a larger self-hosted language model runs on a workstation connected over the local network. This mode increases reasoning capacity while keeping audio, images, and decisions on site. Both modes maintain data locality; selection depends on performance targets and available hardware.

Proposition 2 (Availability Robustness under Hybrid Deployment): *Switching between on-robot and edge-server deployment modes affects inference latency and throughput but does not alter the set of executable functions or the guardrail enforcement layer.*

Proof sketch: Both deployment modes instantiate the same function schema F and validation logic. The on-robot mode runs a 4B-parameter model on the embedded compute, while the edge-server mode runs a 27B-parameter model on a co-located workstation; however, both models receive identical system prompts defining the same five functions (navigate_to, take_picture, analyze_image, speak, activate_safety_protocol) with identical parameter schemas. The command parser and

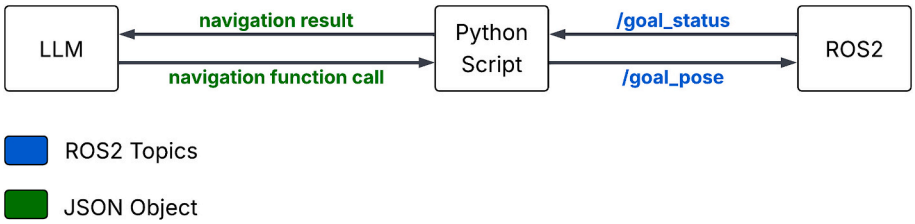


Fig. 2. Communication between LLM and ROS2 for navigation tasks.

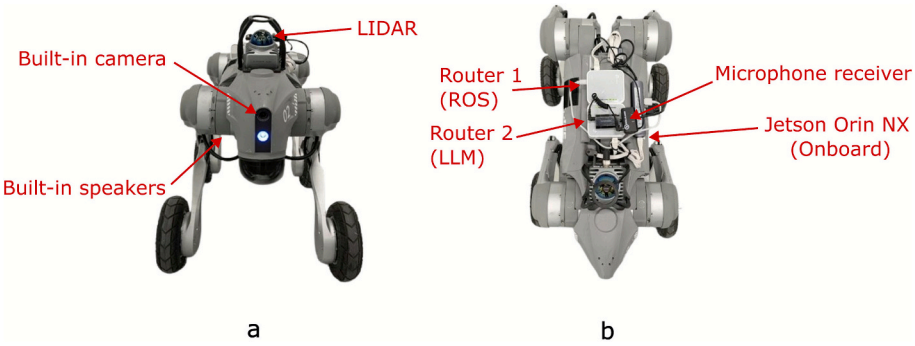


Fig. 3. Robotic platform (Unitree Go2W) and I/O configuration (a) front view and (b) top view.

guardrail layer execute on the robot in both modes, ensuring that validation and safety checks remain invariant. Consequently, the robot's operational capability is preserved regardless of which inference endpoint is active, enabling graceful fallback if the edge server becomes unavailable. Evidence: see the description of both deployment modes above and the identical system prompts constraint in Section 4.1.5 (Table 2).

3.6. Navigation workflow

Navigation is expressed as a high-level function call from the model (e.g., navigate_to_waypoint with a waypoint name or pose). Upon validation, the parser invokes the robot's navigation stack to plan and execute motion. The stack reports meaningful state transitions, such as *en route*, *reached*, *blocked*, or *aborted*, which are surfaced to the model and, in turn, communicated to the user. If navigation fails or becomes unsafe, the assistant can notify the user, request an alternative waypoint, or suggest manual intervention. This loop keeps the operator informed while preserving the conversational paradigm.

3.7. Privacy, safety, and reliability considerations

Data remains on site in both deployment modes. Only the minimum necessary signals are exchanged between components (e.g., transcripts and function outputs), and sensitive artifacts (audio, images) are processed locally. Safety is emphasized through bounded function schemas, argument validation, optional confirmations for movement, and clear status reporting. Reliability is enhanced by deterministic decoding, retries for transient errors, and graceful fallback to dialogue when ambiguity is detected.

Table 1
Computing and inference platforms details.

Deployment	Hardware	OS/Stack	Model	Notes
Robot (Onboard)	NVIDIA Jetson Orin NX (16 GB)	Ubuntu 20.04.5 + JetPack 5.1.1; Ollama 0.10.1	gemma3:4b	Power/ thermals: nvpmodel = MAXN; jetson_clocks enabled; GPU backend via Ollama defaults
Desktop (Co-located server)	RTX 4090 (24 GB), 192 GB RAM, Ryzen 97950X3D	Ubuntu 22.04.5; NVIDIA driver 550.144.03; Ollama 0.11.2	gemma3:27b	GPU backend via Ollama defaults (CUDA); same-LAN access

4. Experimentation and results

This section describes the experimental setup, including the robotic platform, computing infrastructure, and evaluation protocol, followed by quantitative results and statistical analysis.

4.1. Experimental setup

This section details the robotic platform, sensing and middleware, network and model hosting, and the measurement protocol used to evaluate prompt-to-response latency across four workflows.

4.1.1. Robotic platform and I/O

The robotic base is a Unitree Go2W wheeled-legged robot with built-in camera and speakers. An external wireless microphone was added for audio input. A LIVOX MID360 LiDAR was mounted; a derived 2D LaserScan was used for localization and navigation. Two routers were mounted, one of them handling ROS traffic, and the other exclusively for LLM inference. This is explained in the network topology section below. An overview of the hardware setup is shown in Fig. 3.

4.1.2. Computing, inference platform and model selection

The computing substrates and model hosting are summarized in Table 1.

Extensive research and experimentation were performed in order to choose the right models for this case-study. Several factors were taking into account, such as the model size, the multimodal nature, and their open-weights availability. The Gemma3 models were selected as they provide state-of-the-art performance in multimodal workflows and are available in different sizes, allowing the selection of the largest model that can run on the embedded compute (4B) and the largest model that can run on the co-located server (27B). The model selection is out of the

scope for this paper, so the details about all the tested models have been left out of the study to keep it focused on the major contribution.

4.1.3. Middleware and navigation

Navigation uses the ROS 2 Nav2 stack with AMCL for localization (ROS 2 Foxy on the robot). For testing, Nav2 and AMCL ran on an external laptop (ROS 2 Humble), with the robot interfacing over a separate LAN with the laptop. A prior 2D occupancy map of the lab was constructed using GMapping SLAM (Project-MANAS/slam_gmapping" [2019] 2025 [26]). Some work has been done in tuning the Nav2 parameters to be optimized for the wheel-legged hybrid platform, leveraging wheeled motion in straight lines and leg movement for tight spaces. An example of a generated navigation path is shown in Fig. 7.

4.1.4. Network topology

Connectivity uses two dedicated 5 GHz Wi-Fi routers mounted on the robot: (i) Router-A provides a control link for SSH/interfacing with the external laptop and carries Nav2/AMCL traffic; (ii) Router-B bridges the robot to the lab LAN for access to the desktop 27B host and carries only LLM inference traffic. All nodes share the same laboratory subnet/VLAN during experiments.

4.1.5. LLM and session parameters

Extensive experimentation was performed in order to choose the right parameters for an optimal performance of the LLM. A summary of the LLM parameters and the different session values specific to the experiments are summarized in Table 2.

Each parameter in Table 2 was selected to ensure deterministic and reproducible evaluation conditions. Model selection prioritized (i) multimodal capability (for image understanding), (ii) open-weights availability (for self-hosting), and (iii) hardware compatibility with the two deployment modes (embedded on-robot compute vs. a co-located workstation). The LLM temperature values (0.3 for main interactions, 0.2 for visual analysis) balance response diversity with consistency; lower temperatures reduce randomness, improving reproducibility across runs while allowing sufficient flexibility for natural dialogue. Streaming was disabled to measure prompt-to-full-response latency accurately; with streaming enabled, timing would reflect first-token latency rather than complete response generation time.

Fresh sessions were used for each run to ensure measurement independence, eliminating any carryover effects from prior context. The single "hi" warm-up message ensures consistent model initialization by loading the context window and stabilizing inference timing before the experimental prompt is issued. Identical system prompts across models and runs isolate model-specific performance differences from prompt-induced variation.

ASR (automatic speech recognition) was disabled, with prompts entered as text, to control for speech recognition variability and focus evaluation on the LLM's reasoning and function-calling capabilities.

Table 2
Overview of experiment parameters.

Parameter	Value/Setting/Condition
LLM Temperature	Main: 0.3, Visual: 0.2
Streaming	Disabled (prompt to full response delay)
Session	Fresh session/context window per run; single 'hi' warm up message for each run
System prompts	Identical across models and runs
ASR usage	Disabled, prompts entered as text for experimental control
Measured metric	LLM prompt-to-response latency
Timing location	Measured client-side on the robot
Timing method	Prompt dispatch to last token (end of received response)
Input images	LLM could invoke take picture function; standardized control images were injected to control conditions

Timing was measured client-side on the robot, from prompt dispatch to last token received, capturing the end-to-end LLM inference latency. Input images were standardized by injecting controlled images (Fig. 4) rather than relying on live camera captures, ensuring that visual analysis conditions were identical across runs and models.

4.1.6. Task protocol

The experimental task protocol summarized in Table 3 defines four progressively complex workflows designed to evaluate the LLM's ability to execute structured, multimodal reasoning through sequential functional calls. Each experiment incrementally increases task complexity, from simple waypoint navigation to compound behaviors involving visual analysis and human-robot communication. The corresponding visual inputs used for evaluation are illustrated in Fig. 4 (a and b), which depict compliant (Fig. 4 (a)) and non-compliant (Fig. 4 (b)) scenarios of a worker wearing or not wearing a hard hat, respectively. These images were captured directly by the robot's onboard camera and served as standardized inputs for the *take picture* and *analyze image* functions. Together, the table and figures establish the experimental context for assessing whether the models could interpret visual cues, generate appropriate structured commands, and verbally report findings in accordance with the designed workflow.

4.1.7. Detailed experiment descriptions

Experiment 1 (Simple Navigation): The voice command "Go to the work area" tests basic command interpretation and single-function execution. The expected workflow consists of one step: navigate_to (location = "work_area"). No image analysis is involved. Success is defined as the robot navigating to the work area waypoint and reporting arrival status.

Experiment 2 (Inspection with Report): The voice command "Inspect the work area" tests a multi-step workflow combining navigation, perception, and verbal communication. The expected workflow consists of five steps: (1) navigate_to(location = "work_area"), (2) take_picture(), (3) analyze_image() with prompt "analyze the image", (4) speak(text = findings). The image analysis receives a compliant scene (Fig. 4a, worker wearing hard hat) and the LLM generates a general description of the work area and worker safety status. The robot then vocalizes these findings.

Experiment 3 (Inspection with Return): The voice command "Inspect the work area and report back at the station" extends Experiment 2 with return navigation and station-based reporting. The expected workflow consists of six steps: (1) navigate_to(location = "work_area"), (2) take_picture(), (3) analyze_image(), (4) navigate_to(location = "station"), (5) speak(text = findings). The image analysis is identical to Experiment 2 (compliant scene). This experiment tests the LLM's ability to sequence multiple navigation goals and defer verbal output to a specific location.

Experiment 4 (Safety Violation Response): The voice command "Inspect the work area" (identical to Experiment 2) is issued, but the injected image shows a non-compliant scene (Fig. 4b, worker not wearing hard hat). The expected workflow consists of thirteen steps testing complex conditional reasoning and multi-party dialogue: (1) navigate_to(location = "work_area"), (2) take_picture(), (3) analyze_image() detecting the PPE violation, (4) speak() to notify the worker about the violation, (5) dialogue to confirm hard hat fetch request, (6) navigate_to(location = "storage_area"), (7) speak() to interact with storage manager, (8) dialogue to confirm hard hat retrieval, (9) navigate_to(location = "work_area"), (10) speak() to deliver hard hat with safety guidance, (11) navigate_to(location = "station"), (12–13) speak() to report findings and resolution to supervisor. This experiment evaluates whether the LLM can detect safety violations through image analysis and execute an appropriate corrective workflow.

4.1.8. Functions available to the LLM

The assistant operated through a limited set of well-defined functions

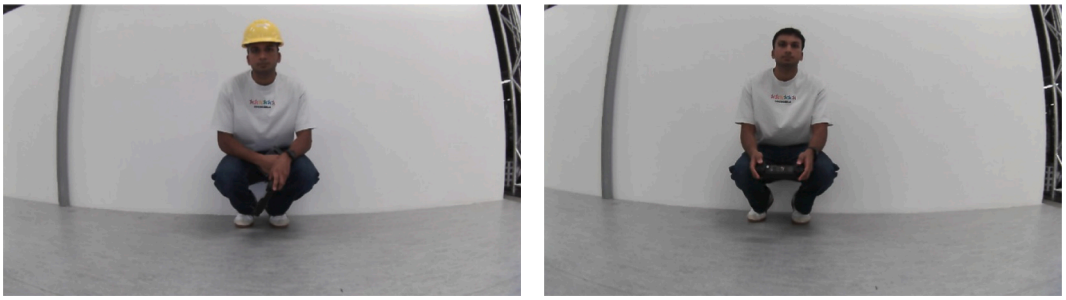


Fig. 4. (a) Compliant image of a worker wearing a hard hat and (b) non-compliant image of a worker not wearing a hard hat taken with the robot camera.

Table 3
Summary of the task protocols for each experiment.

Experiment	Input prompt	Injected image	Expected steps (high-level)
1	Go to the work area.	Compliant (Fig. 4(a))	- navigate to work area
2	Inspect the work area.	Compliant (Fig. 4(a))	- navigate to work area - take picture - analyze image - speak findings
3	Inspect the work area and report back at the station.	Compliant (Fig. 4(a))	- navigate to work area - take picture - analyze image - navigate back to station - speak
4	Inspect the work area	Non-compliant (Fig. 4 (b))	- navigate to work area - take picture - analyze image - notify worker about violation - confirm hard hat fetch request - navigate to storage area - interact with storage manager - retrieve hard hat and confirm it is securely attached - navigate to work area - deliver hard hat to worker with safety tip - navigate to station - speaking findings and resolution to supervisor

exposed to the language model, each representing a discrete and verifiable robot capability. These included:

- *navigate_to(location)*: Navigates to predefined waypoints, such as the work area, storage area, or designated station, and returns an execution status indicating whether the robot is en route, has failed, or has successfully reached the target location.
- *take_picture()*: Activates the robot's camera pipeline to capture an image of the current environment and provides a reference to the captured image for subsequent tasks.
- *analyze_image()*: Invokes the multimodal LLM's native vision capabilities within a dedicated model context. When called, the captured image is passed to the same Gemma 3 model used for dialogue, along with a task-appropriate prompt (e.g., "analyze the image" for general inspection or a safety-focused prompt for PPE compliance checks). The LLM processes the visual input and generates a natural language description of the scene.
- *speak(text)*: Passes the generated textual output to the text-to-speech (TTS) module, producing speech synthesis in a sound file which is then played through the robot speakers
- activate_safety_protocol(): When the LLM detects a safety violation during image analysis (e.g., missing PPE), it can call this function to load a specialized safety compliance prompt. This prompt provides the LLM with a structured workflow for addressing the violation, including steps for worker notification, equipment retrieval, and incident reporting. This mechanism enables context-aware behavior switching without requiring separate fine-tuned models for each safety scenario.

The analyze_image() function leverages the native multimodal capabilities of the selected LLM model rather than relying on a separate computer vision pipeline. The same model (either 4b or 27b) handles both visual interpretation and dialogue/function calling. For example, in Experiment 4 (safety compliance inspection), when the robot captures an image of a worker, the LLM receives the image with the prompt "analyze the image." The model then generates descriptive output such as: "The worker is wearing appropriate safety attire, including a hard hat, high-visibility vest, and safety boots. The work area appears organized with tools properly stored." Alternatively, for a non-compliant scenario: "The worker is not wearing a hard hat. This is a safety violation that should be addressed immediately."

Each function call followed a structured JSON schema that bound arguments and responses, enabling the system to verify syntax, validate parameters, and log all actions prior to execution. This structured function-calling mechanism ensured transparency, safety, and modular integration with the ROS 2 middleware while allowing the LLM to chain perception, reasoning, and navigation behaviors into coherent task sequences.

Example JSON structure for function calling:

```json
{
"function": "navigate_to",
"params": {"location": "workbench"},
"speak": "Heading to the workbench now."
}
```

This example JSON output from the LLM would trigger the navigation to the workbench while playing "Heading to the workbench now" on the robot speaker.

4.1.9. Experimental design

The experimental design comprises a full factorial arrangement of two models (27B and 4B) and four experiments, yielding eight model-condition combinations. For each combination, five independent runs

were conducted, resulting in 40 total observations (2 models × 4 experiments × 5 runs). Each run used a fresh session to ensure independence, while prompt templates remained identical across all conditions. A uniform single-message warm-up policy was applied throughout.

Outcome labels were assigned through manual inspection of execution logs after each run. Each executed step was classified as: success (the step was executed correctly and aligned with the planned workflow), extra (an unnecessary step that was not required by the plan but did not negatively affect the overall workflow, such as a redundant speak command or an unplanned intermediate navigation), or failure (an incorrect or incomplete execution, such as a malformed JSON output, wrong function call, or incomplete workflow). Failed steps were excluded from timing metrics to avoid conflating execution errors with inference performance.

Regarding model differentiation: the 27B model (Gemma 3 27B) has approximately 6.75 times more parameters than the 4B model (Gemma 3 4B). The 4B model runs on the robot's embedded compute (Jetson Orin NX, 16 GB memory), while the 27B model runs on a co-located edge server (desktop workstation with RTX 4090, 24GB VRAM for inference). Both models share the same architecture family and were prompted identically; the key difference lies in parameter number (dependent on the model) and inference speed (dependent on the hardware).

4.1.10. Experiment area and navigation waypoints

The three designated waypoints (i.e., (a) station (b) storage area and (c) work area) were located on a generated map of the space used to conduct the experiments (Fig. 5). The map was generated using the GMapping SLAM package ("Project-MANAS/slam_gmapping" [2019] 2025).

The coordinates of those locations as well as the robot pose was coded into the *navigate_to()* function. The LLM could call the navigation to these locations at a high level, but did not have low-level access to modify or view the location coordinates and robot pose.

Lemma 3 (Robustness to Invalid Navigation Commands): *The navigation subsystem rejects any navigate_to call whose location parameter does not match a predefined waypoint identifier, preventing arbitrary or unsafe navigation commands.*

Proof sketch: The three designated waypoints (station, storage_area, work_area) are defined in the system configuration with fixed (x, y, theta) coordinates. The navigate_to function accepts only these string identifiers as valid inputs; the LLM cannot specify raw coordinates or modify existing waypoint poses. When the LLM generates a navigate_to call, the command parser validates the location parameter against the set {station, storage_area, work_area}. If the parameter is absent, malformed, or does not match any predefined identifier, the call is rejected and no motion is initiated. This design ensures that navigation is constrained to the verified, safe regions shown in Fig. 5, regardless of LLM hallucinations or adversarial prompts. Evidence: see the waypoint definitions in Fig. 5, the coordinate encoding described above, and the limited function exposure in Section 4.1.7.

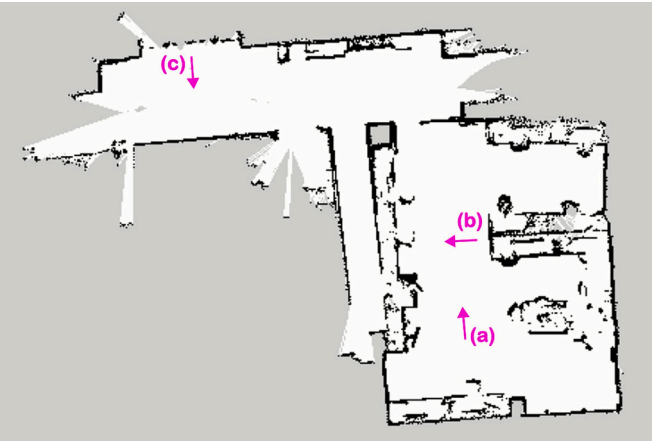


Fig. 5. Generated map of the laboratory space with labeled waypoints poses (a) station, (b) storage area, and (c) work area.

Fig. 6 shows the robot at the different locations (station, storage area, and work area, respectively) during ongoing experimentation, and Fig. 7 shows a generate path from the station to the work area.

4.2. Results

The following subsections present the evaluation metrics, sample size validation, and comparative analysis between the two deployment configurations.

4.2.1. Metrics definition and validation

Five key performance metrics were established to evaluate model performance across experiments:

A. Mean delay per step: Average LLM prompt-to-response latency per step within a run
B. Total run delay: Cumulative LLM latency for complete runs (experiments 1–2 only, where all steps completed successfully)
C. Success proportion: Ratio of successful steps to non-extra steps
D. Extra step rate: Proportion of unnecessary steps beyond the planned workflow
E. Total steps: Number of steps executed per run

The metrics are directly derived from the outcome labels assigned during log inspection:

Success Proportion is calculated as the number of steps labeled "success" divided by the number of non-extra steps (i.e., success / (success + failure)). This metric excludes extra steps from both numerator and denominator because extra steps, while unnecessary, do not represent execution failures.

Extra Step Rate is calculated as the number of steps labeled "extra" divided by the total number of steps executed (i.e., extra / (success + extra + failure)). This metric captures workflow inefficiency due to redundant actions.

Examples of extra steps observed in the experiments include: redundant speak commands where the LLM verbalized information already communicated, unplanned intermediate navigations to waypoints not required by the workflow, and redundant clarification turns. Examples of failure modes include: malformed JSON outputs that failed schema validation, incorrect function calls (e.g., calling speak when navigate_to was expected), and incomplete workflows where the LLM terminated before completing all required steps.

For timing metrics (mean delay per step, total run delay), only successful steps are included. Steps labeled as failure are excluded because their timing may reflect error handling rather than typical inference performance. This approach ensures that delay metrics characterize normal operational performance rather than being skewed by exceptional error cases.

Proposition 4 (Graceful Degradation under LLM Errors): *When the LLM produces suboptimal outputs (malformed JSON, incorrect function calls, or unnecessary steps), the system degrades gracefully: failures are logged as reduced success proportion, unnecessary actions are logged as extra steps, and no unsafe execution occurs.*

Proof sketch: The outcome labeling scheme classifies each executed step as success (correct execution aligned with the planned workflow), extra (an unnecessary but non-harmful step, such as a redundant speak command or an unplanned navigation), or failure (an incorrect or incomplete action, such as a malformed JSON output or wrong function call). Critically, the validation and guardrail layer prevents any action that violates schema constraints from executing; such attempts are recorded as failures without physical effect. Extra steps consume time but do not compromise safety because they still satisfy the bounded



Fig. 6. General view of the robotic platform at the (a) station, (b) storage area, and (c) work area during experimentation.

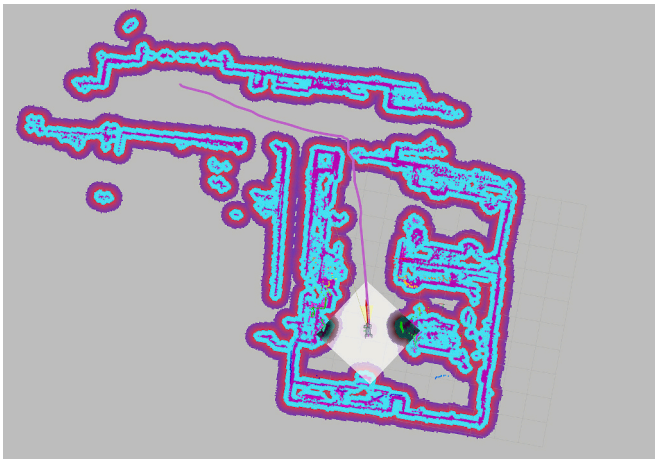


Fig. 7. Navigation path from the station to the work area prioritizing straight lines to leverage the wheeled motion of the Go2W platform.

action space property (Lemma 1). The experimental results in Table 6, Table 7, and Table 8 demonstrate this graceful degradation: the 4B model exhibited reduced success rates (83.3% and 76.5% in Experiments 3 and 4) and elevated extra step rates (14.3% and 10.5%), yet no unsafe behaviors were observed. Evidence: see the metric definitions above, the outcome labels in Section 4.1.8, and Tables 6–8 in Section 4.2.3.

4.2.2. Sample size validation via SESOI analysis

The objective of this analysis is to validate that five runs per condition provide sufficient measurement precision to support practical deployment decisions. With limited experimental resources, it is essential to confirm that observed timing differences are not artifacts of sampling variability but reflect genuine performance characteristics.

To validate that five runs per condition provide sufficient precision for delay metrics, a Smallest Effect Size of Interest (SESOI) analysis was conducted using a threshold of 0.5 s per step. This threshold represents a practically meaningful difference in human-machine interaction latency; response delays below this threshold are generally imperceptible or acceptable to users, while delays exceeding it may disrupt natural dialogue flow.

The Student's t-distribution was employed for confidence interval estimation because it is specifically designed for small sample sizes (here, $n = 5$, yielding df $= 4$). Unlike the normal distribution, the t-distribution has heavier tails that appropriately account for the increased uncertainty when estimating population parameters from few observations. Using the normal distribution would underestimate confidence interval widths and risk false precision.

As a complementary validation, bootstrapping was applied, to construct non-parametric confidence intervals. Bootstrapping does not assume normality of the underlying distribution and provides a robust check on the parametric t-distribution results. Agreement between both methods strengthens confidence that the sample size is adequate. For delay metrics, all conditions demonstrated confidence intervals well within the SESOI bounds:

- 27B model: All experiments showed t-distribution 95% CI half-widths $\leq$0.26 s, with coefficients of variation ranging from 1.5% to 6.9%
- 4B model: All experiments showed t-distribution 95% CI half-widths $\leq$0.28 s, with coefficients of variation ranging from 0.9% to 3.6%
- Bootstrap validation: Bootstrap 95% confidence intervals consistently fell within SESOI bounds, confirming the adequacy of n = 5 runs

The precision analysis confirms that five runs provide sufficient statistical power to detect meaningful differences while maintaining practical experimental efficiency. For success proportion, extra step rate, and total steps, no validation was required as they were invariant across runs. Fig. 8 shows the mean value over the five runs for each of the metrics, illustrating their variation across experiment numbers and model conditions. The metrics include (a) Mean Delay per Step, (b) Total Run Delay (available for Experiments 1–2 only), (c) Success Proportion, (d) Extra Step Rate, and (e) Total Steps. Error bars indicate $\pm$1 standard deviation, and individual run data points are shown with transparency. The dataset comprises 40 total observations (4 experiments $\times$ 2 models $\times$ 5 runs per condition).

4.2.3. Model comparison

The objective of this section is to quantify whether performance differences between the 27B and 4B models are statistically significant and practically meaningful, or whether observed differences could plausibly arise from measurement noise.

One-way ANOVA (Analysis of Variance) was employed to compare delay metrics between models. ANOVA is appropriate here because two groups (27B vs. 4B) are being compared on continuous outcome variables (delay times), and ANOVA provides a formal test of whether group means differ beyond what would be expected by chance. For two-group comparisons, ANOVA is equivalent to the independent-samples *t*-test but generalizes naturally if additional model conditions are introduced in future work.

The F-statistic produced by ANOVA represents a signal-to-noise ratio: it is the ratio of between-group variance (differences attributable to the model) to within-group variance (measurement noise and run-to-run variability). An F-value of 1 indicates that between-group

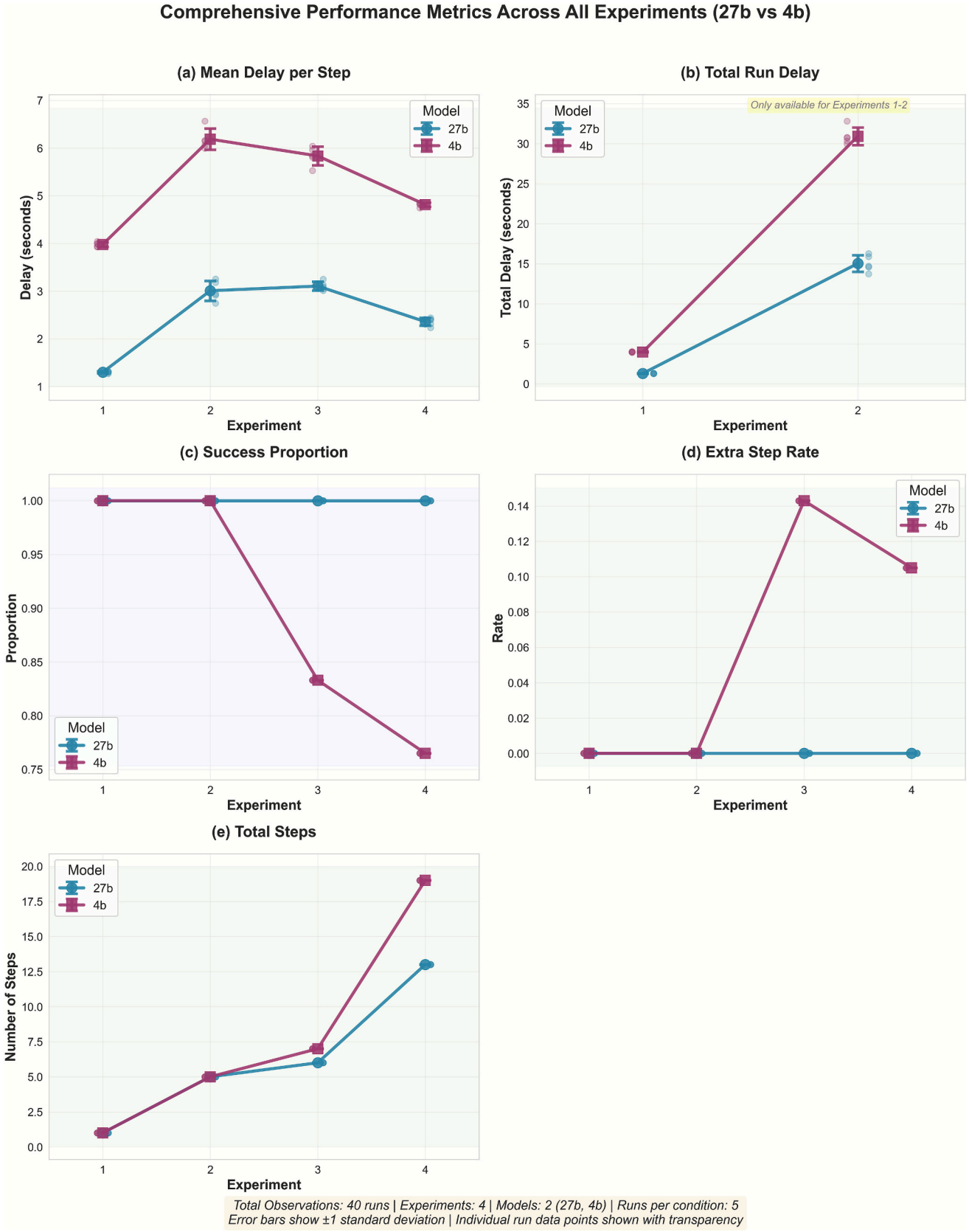


Fig. 8. Plots of each metric means across models and experiments showing (a) mean delay per step, (b) total run delay, (c) success proportion, (d) extra step rate, and (e) total steps.

differences are no larger than random variation; F-values substantially greater than 1 indicate that model differences exceed measurement noise. As a benchmark, F-values above 10 are typically considered large effects in behavioral research.

To avoid conflating task complexity with model effects, separate ANOVAs were conducted for each experiment (Experiments 1–4), comparing the two models within each experimental context. This approach accounts for the fact that different experiments involve different task complexities and expected baseline delays.

Interpretation of reported F-values: In Table 4 and Table 5, the F-statistics range from 13,430 to 548. These extraordinarily large values indicate that the variance attributable to model differences is thousands of times larger than the within-model run-to-run variance. For example, F = 13,430 in Experiment 1 means that the model effect is 13,430 times larger than measurement noise. Such values provide overwhelming statistical evidence that the observed performance differences are genuine rather than artifacts of sampling. The corresponding *p*-values (all below 10^-8) confirm that the probability of observing such differences by chance, if the models were truly equivalent, is negligibly small.

Table 4 presents mean delay per step for each experiment, showing that the 27B model consistently outperformed the 4B model with time savings ranging from 2.45 to 3.18 s per step (all $p < 0.001$). Table 5 extends this analysis to total run delay for Experiments 1 and 2, where both models completed all runs successfully, revealing cumulative time advantages of 2.68 and 15.89 s respectively for the 27B model.

It is worth noting that the total run delay analysis is limited to experiments 1 and 2, where all runs completed successfully for both models without failed steps.

For metrics showing no variation across runs, simple deltas are reported rather than statistical tests. Table 6, Table 7, and Table 8 present comparisons of success proportion, extra step rate, and total steps respectively. While both models achieved perfect performance in simple workflows (Experiments 1–2), the 4B model's performance degraded substantially in complex scenarios. In Experiments 3 and 4, the 27B model maintained 100% success with no extra steps, while the 4B model exhibited reduced success rates (83.3% and 76.5%), performed unnecessary extra steps (14.3% and 10.5%), and required significantly more total steps to complete tasks (1 and 6 additional steps respectively).

In order to understand how execution time was distributed across different task types within each workflow, the time composition was analyzed by step for all experiments. Fig. 9 and Fig. 10 present the breakdown for Experiments 1 and 2, illustrating how the 27B model's superior performance translates to faster completion across all step types in simple workflows.

Fig. 11 and Fig. 12 extend this analysis to the more complex Experiments 3 and 4, respectively, where the increased diversity of step types and the emergence of extra unnecessary steps in the 4B model (indicated by hatched bars) demonstrate the compounding effect of both slower individual operations and extra recovery attempts required by the smaller model.

To better understand how model performance varied across different operation types, separate analyses were conducted for navigation and image processing tasks. Fig. 13 presents navigation performance across all experiments, comparing execution times for pure navigation-only steps while excluding combined actions to isolate navigation-specific capabilities. Fig. 14 illustrates performance for image-related operations (including both camera capture and image interpretation) across Experiments 2, 3, and 4. These task-specific analyses reveal distinct performance patterns, with the 27B model demonstrating variable advantages in navigation tasks but complete dominance in image processing operations, as detailed in the following subsections. This analysis is useful when considering a hybrid approach between both an on-device model and a co-hosted larger model.

An example log of the entire interaction sequence for a run of Experiment 4 can be found in the Appendix.

Table 4
Mean delay per step (All experiments).

Experiment	27B Mean (s)	4B Mean (s)	Delta (27B—4B)	F-statistic	p-value
1	1.30	3.98	−2.68	13,430	3.4×10^{-14}
2	3.01	6.19	−3.18	548	1.2×10^{-8}
3	3.11	5.83	−2.73	805	2.6×10^{-9}
4	2.36	4.81	−2.45	3516	7.3×10^{-12}

Table 5
Total run delay (Experiment 1&2 only).

Experiment	27B Mean (s)	4B Mean (s)	Delta (27B—4B)	F-statistic	p-value
1	1.30	3.98	−2.68	13,430	3.4×10^{-14}
2	15.04	30.93	−15.89	548	1.2×10^{-8}
3	18.638	–	–	–	–
4	30.678	–	–	–	–

Table 6
Comparison of success proportion.

Experiment	27B value	4B value	Delta (27B—4B)	Notes
1	100%	100%	0.0	Both models achieved 100% success
2	100%	100%	0.0	Both models achieved 100% success
3	100%	83.3%	+0.17%	27B maintains perfect execution
4	100%	76.5%	+0.24%	27B maintains perfect execution

Table 7
Comparison of extra step rate.

Experiment	27B value	4B value	Delta (27B—4B)	Notes
1	0%	0%	0.0	No extra steps for either model
2	0%	0%	0.0	No extra steps for either model
3	0%	14.3%	14.3%	4B performs unnecessary steps
4	0%	10.5%	10.5%	4B performs unnecessary steps

Table 8
Comparison of total steps.

Experiment	27B value	4B value	Delta (27B—4B)	Notes
1	1	1	0	Identical step counts
2	5	5	0	Identical step counts
3	6	7	−1	4B requires one additional step
4	13	19	−6	4B requires 46% more steps

5. Discussion of results

This section interprets the experimental findings, examines statistical robustness, and discusses implications for deployment strategies in

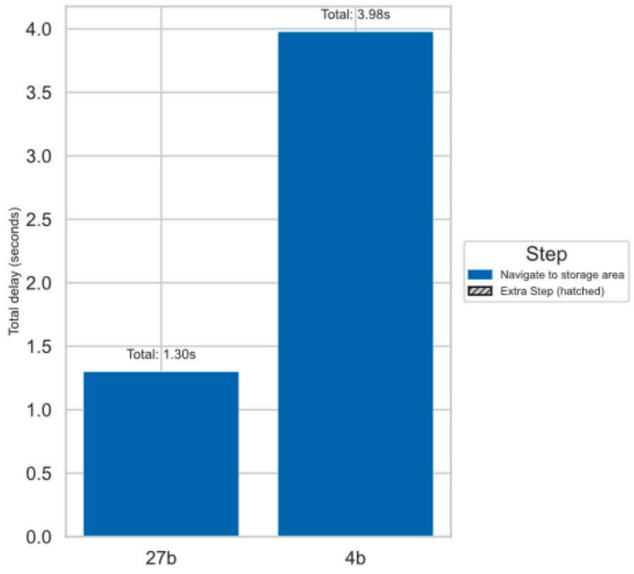


Fig. 9. Time composition of experiment 1 by step.

construction robotics.

Theorem 5 (Robustness to Increasing Task Complexity): *The 27B edge-server model maintains consistent execution reliability (100% success proportion, 0% extra step rate) across all tested workflow complexities, from single-step navigation (Experiment 1) to multi-step compound workflows involving navigation, perception, dialogue, and corrective actions (Experiment 4).*

Proof sketch: Table 3 defines four experiments with progressively increasing complexity: Experiment 1 requires one step (navigate), Experiment 2 requires five steps (navigate, take picture, analyze image, speak findings), Experiment 3 requires six steps (adding return navigation and station report), and Experiment 4 requires thirteen steps (including violation detection, dialogue with worker, hard hat retrieval, and supervisor report). For the 27B model, Tables 6 and 7 show that success proportion remains at 100% and extra step rate remains at 0% across all four experiments, indicating that task complexity does not degrade execution reliability. In contrast, the 4B model shows degradation beginning at Experiment 3 (83.3% success, 14.3% extra) and worsening at Experiment 4 (76.5% success, 10.5% extra), demonstrating that smaller model capacity is a limiting factor under increased complexity. The 27B model's robustness arises from its larger reasoning capacity, which enables accurate multi-step planning, correct function sequencing, and avoidance of redundant or erroneous actions. Evidence: see Table 3 (workflow definitions), Table 6 (success proportion), Table 7 (extra step rate), and Figs. 9–12 (time composition by step). □.

5.1. Statistical analysis approach

The analysis employed a dual statistical strategy appropriate for data characteristics: formal ANOVA testing for delay metrics with measurable variance, and simple delta reporting for invariant metrics where one or both models showed constant performance. This approach avoids inappropriate statistical testing of invariant data while providing comprehensive performance characterization.

The SESOI validation demonstrates that the experimental design achieves adequate precision for practical decision-making. The 0.5-s threshold represents a meaningful difference in human-robot interaction contexts, and the confidence intervals consistently fall well below this threshold.

5.2. Performance interpretation

Based on the results shown in Fig. 8 the 27B model (co-located server) demonstrates consistent advantages across all metrics and complexity levels. In Fig. 8(a) (Mean Delay per Step), the 27B model exhibits substantially lower and more stable step-level latency across all experiments, indicating more efficient response generation and task

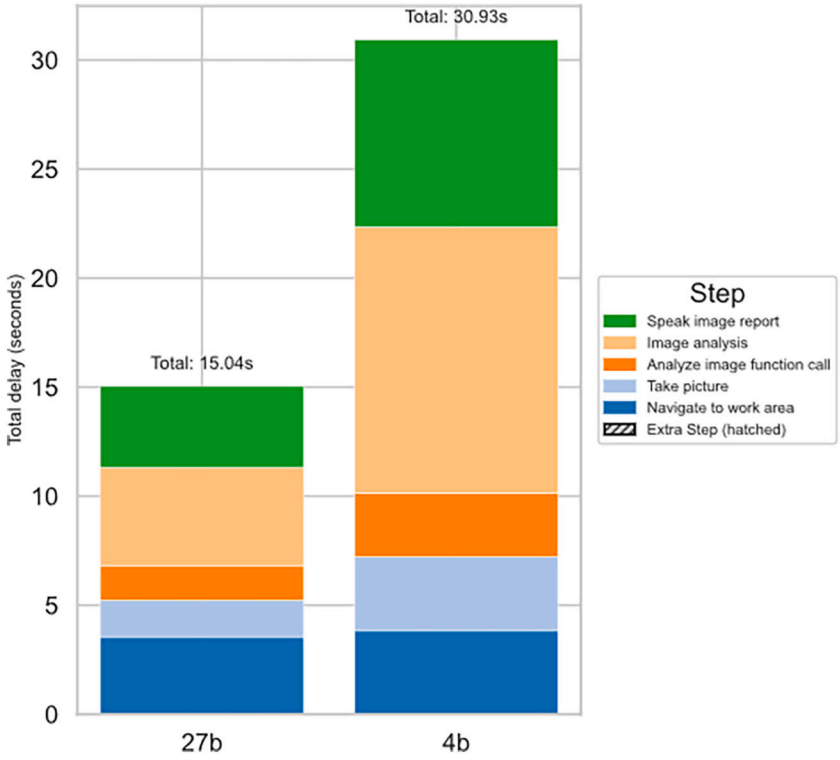


Fig. 10. Time composition of experiment 2 by step.

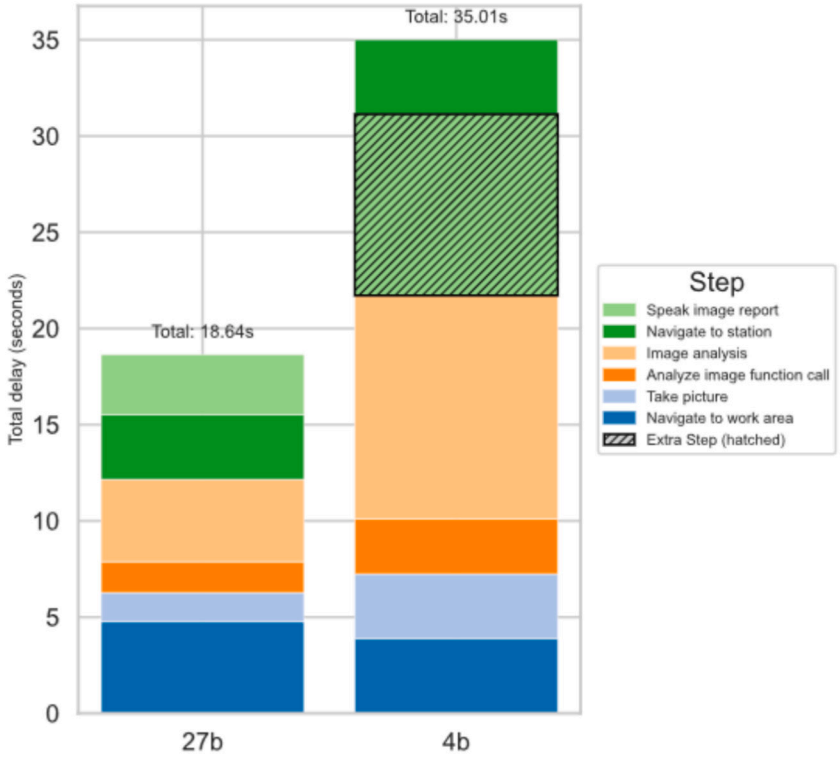


Fig. 11. Time composition of experiment 3 by step.

execution. The 4B model shows a latency increase in Experiment 2 that persists across later experiments, suggesting a sensitivity to workflow complexity. Similarly, Fig. 8(b) (Total Run Delay, Experiments 1 and 2 only) shows that cumulative task completion times are markedly shorter for the 27B model, reflecting overall time efficiency under identical operational conditions.

Fig. 8(c) (Success Proportion) highlights the reliability contrast. The 27B model maintains near-perfect task success across all experiments, whereas the 4B model's performance declines significantly in Experiments 3 and 4, corresponding to increased workflow complexity. In Fig. 8(d) (Extra Step Rate), the 4B model exhibits a higher frequency of unnecessary steps, particularly in Experiment 3, while the 27B model maintains negligible rates throughout. This pattern indicates greater task adherence and control stability for the 27B configuration.

Finally, Fig. 8(e) (Total Steps) shows that both models execute more steps as task complexity increases; however, the 4B model consistently performs additional steps beyond the planned workflow, reflecting compensatory or corrective actions during execution. These additional actions contribute to longer run times and indicate deviations from optimal task flow. This also shows that the 27B model is effective at combining tasks that can occur in parallel, e.g. 'Confirm hard hat request and navigate to storage area' as shown in Fig. 12.

Collectively, the step-count differences and timing results suggest that the 4B model's extended execution durations in complex workflows (Experiments 3–4) arise from both slower per-step performance and a higher incidence of recovery or redundant behaviors. In contrast, the 27B model sustains stable timing, minimal deviations, and high task completion fidelity across all tested conditions, underscoring its robustness under increasing task demands.

5.3. Statistical robustness

The statistical evidence for performance differences between models is exceptionally strong. The F-statistics reported in Table 4 and Table 5 range from 548 to 13,430, with corresponding p-values below 10^-8 (ranging from 1.2×10^{-8} to 3.4×10^{-14}). To contextualize these values: in behavioral and applied research, F-values above 10 are typically interpreted as large effects. The minimum observed F-value (548 in Experiment 2 total run delay) exceeds this threshold by a factor of 55, and the maximum (13,430 in Experiment 1) exceeds it by over 1300 times. This indicates that the model-attributable variance dwarfs measurement noise across all tested conditions.

The F-statistic can be interpreted as a signal-to-noise ratio. An F-value of 548 means that the between-model variance is 548 times larger than the within-model run-to-run variance; an F-value of 13,430 means the signal is over 13,000 times stronger than the noise. Such magnitudes leave no ambiguity about the reliability of observed differences.

The consistency of results across all four experiments further strengthens confidence. The 27B model outperformed the 4B model in every condition, with no reversals or anomalous results. This consistency across varying task complexities (from single-step navigation to thirteen-step compound workflows) suggests that the performance advantage is robust and generalizable rather than task-specific.

These statistical findings complement the architectural robustness guarantees formalized earlier (Lemma 1, Proposition 2, Lemma 3, Proposition 4, and Theorem 5). Together, they demonstrate that the system's robustness is both structurally enforced by design (bounded action space, validated parameters, graceful degradation) and empirically confirmed by reproducible quantitative experiments with overwhelming statistical confidence.

5.4. Task complexity and execution reliability

The results reveal distinct performance patterns as workflow complexity increases. In simple workflows (Experiments 1–2), both models achieved perfect success rates (100%) with no extra steps, though the 27B model demonstrated significantly lower latency per step. As complexity increased in Experiment 3, the 27B model maintained perfect execution (100% success, 0% extra steps) while the 4B model began showing degradation (83.3% success, 14.3% extra steps, requiring one additional step on average). This divergence became pronounced in the high-complexity Experiment 4, where the 27B model

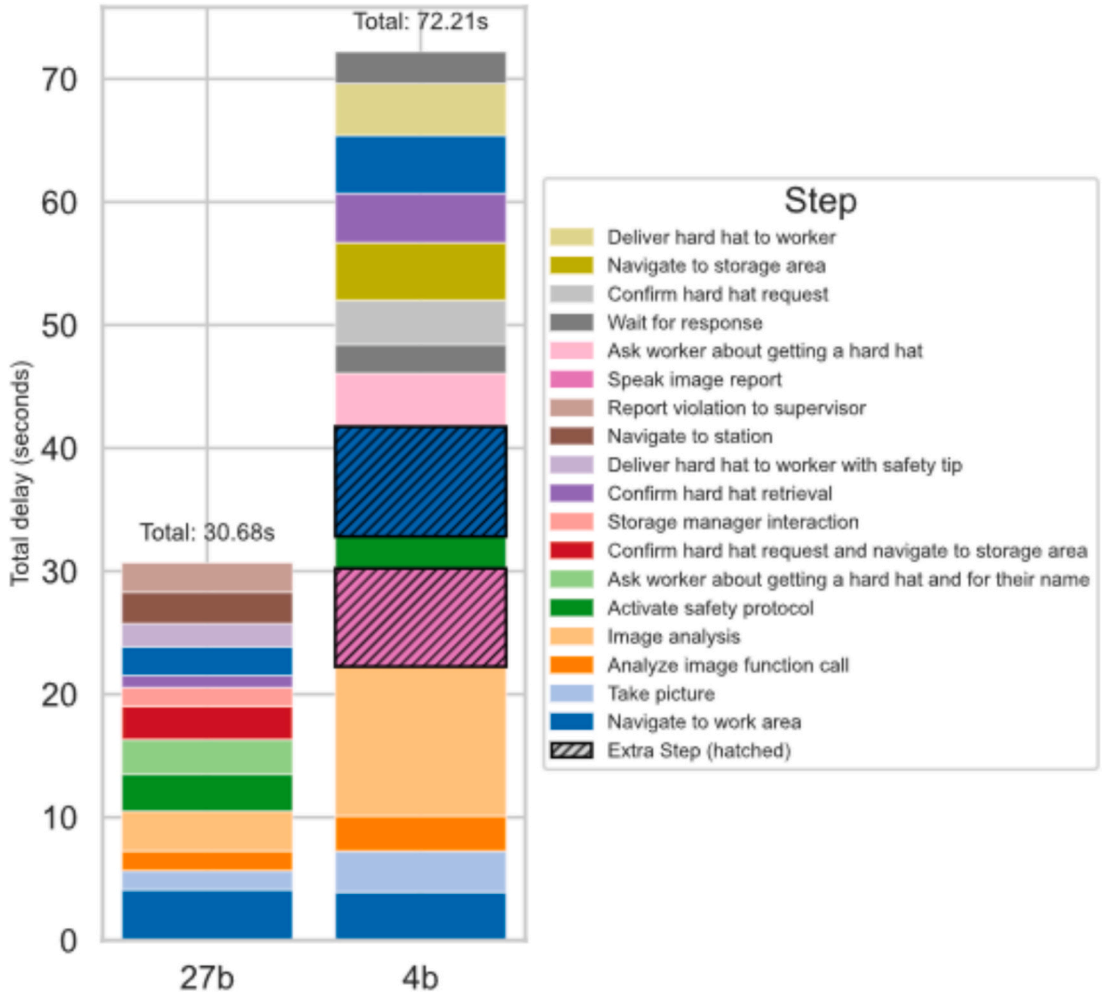


Fig. 12. Time composition of experiment 4 by step.

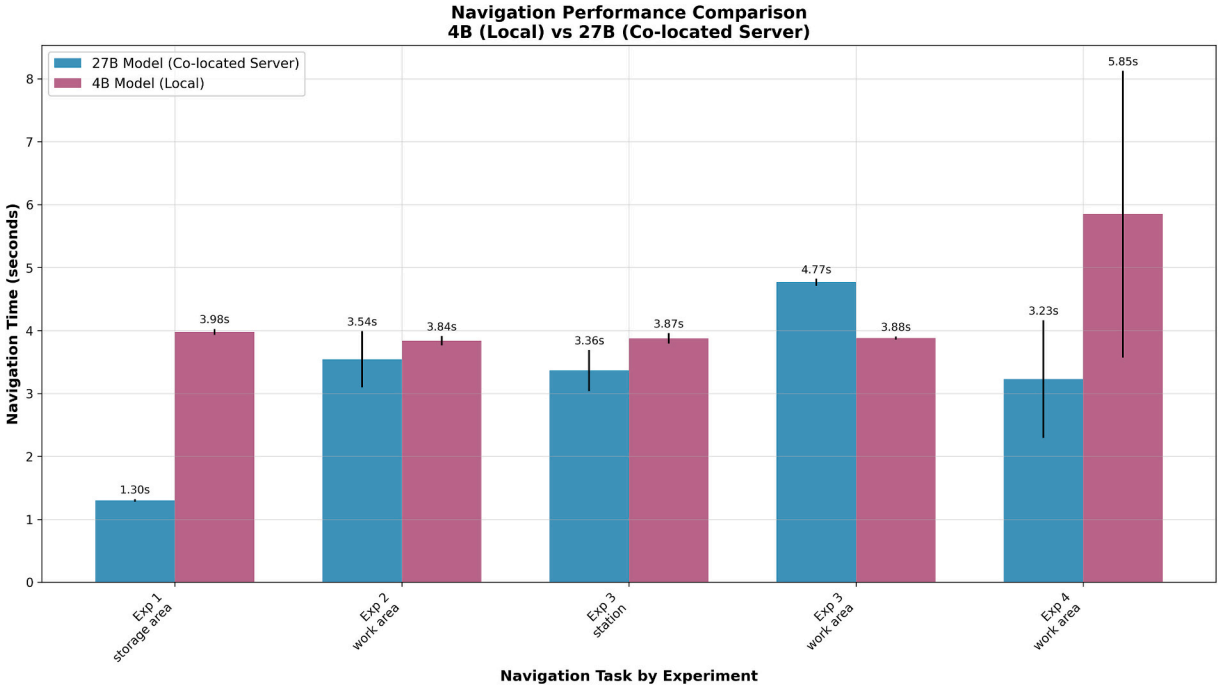


Fig. 13. Delays of navigation tasks across experiments.

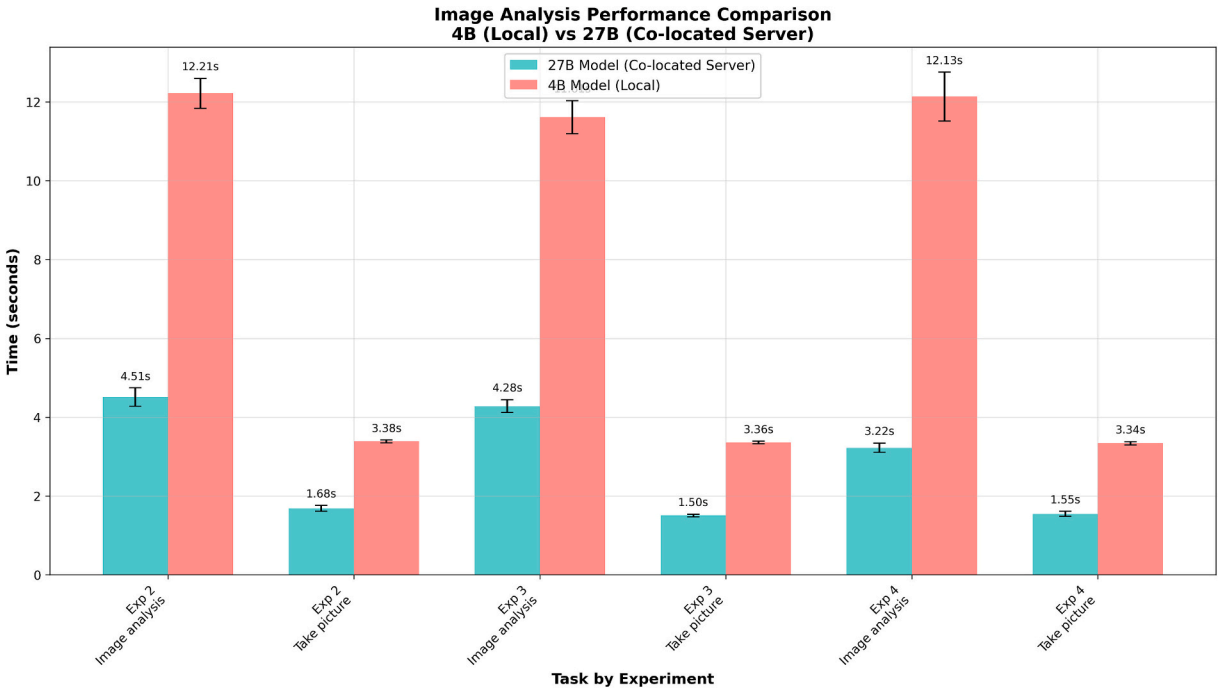


Fig. 14. Delays of image analysis tasks (image analysis and take picture) across experiments.

maintained 100% success with no extra steps, while the 4B model achieved only 76.5% success with 10.5% extra steps, requiring 46% more total steps (19 vs 13). The time composition of experiments as show in Fig. 9 to Fig. 12 also show that for the simpler experiments 1 and 2, the steps composition is the same across models, with significant differences showing up for the more complex experiments 3 and 4. In Fig. 12 specifically for experiment 4, a significant disparity can be seen between in efficiency between the two models.

The step count differences in complex workflows indicate that the 4B model's longer execution times result not only from slower individual steps but also from additional recovery attempts and workflow deviations. This compounds the latency disadvantage and suggests reduced robustness in challenging scenarios requiring multi-step reasoning and error recovery.

5.5. Navigation performance analysis

To investigate task-specific performance characteristics, pure navigation-only steps were analyzed across all experiments, excluding combined actions to isolate navigation-specific capabilities (see Fig. 13). Overall, the 27B model demonstrated superior navigation performance (3.24 s average vs 4.73 s for 4B), winning in four of five navigation scenarios (80%). However, the 4B model showed one significant advantage in Experiment 3's "Navigate to work area" task (3.88 s vs 4.77 s, 18.6% faster, $p < 0.001$, Cohen's d = 19.86), with superior consistency ($\sigma = 0.03$ s vs 0.06 s).

Detailed experiment-by-experiment results showed: Experiment 1 navigation to storage area favored 27B by 206% (1.30s vs 3.98 s, $p < 0.001$); Experiment 2 navigation to work area showed no significant difference (3.54 s vs 3.84 s, $p = 0.186$); Experiment 3 navigation to station favored 27B by 15.1% (3.36 s vs 3.87 s, $p = 0.010$); Experiment 3 navigation to work area favored 4B by 18.6% (3.88 s vs 4.77 s, $p < 0.001$); and Experiment 4 navigation to work area favored 27B by 81.2% (3.23 s vs 5.85 s, $p = 0.002$).

5.6. Image analysis performance

Image analysis performance was evaluated across Experiments 2, 3, and 4, focusing on camera capture ("Take Picture") and image interpretation ("Image Analysis") (see Fig. 14). The 27B model demonstrated complete dominance across all image-related tasks, with an overall performance advantage of 174.9% (2.79 s average vs 7.67 s for 4B, all comparisons $p < 0.001$). For the "Take Picture" task, the 27B model averaged 1.58 s ($\sigma = 0.09$ s) compared to 4B's 3.36 s ($\sigma = 0.04$ s), representing a 112.9% performance advantage across all three scenarios. For "Image Analysis," the 27B model averaged 4.00s ($\sigma = 0.65$ s) compared to 4B's 11.98 s ($\sigma = 0.31$ s), representing a 199.4% performance advantage.

Unlike navigation tasks where the 4B model showed competitive performance in specific scenarios, the 27B model won all image-related comparisons with substantial performance gaps ranging from 101% to 276%. While the 27B model showed higher variability in image analysis compared to 4B, this variability occurred at much faster baseline performance levels, making the trade-off favorable. The larger performance gaps in image analysis compared to navigation suggest that the 27B model's advantages are more pronounced in cognitively demanding tasks requiring visual processing and interpretation.

5.7. Implications for hybrid deployment strategies

Comprehensive performance analysis revealed opportunities for task-aware hybrid architectures that leverage each model's strengths while maintaining operational resilience. The 27B model's 174.9% performance advantage in image tasks makes it the optimal choice for all visual processing workflows. For navigation, the 27B model demonstrates superior overall performance but the 4B model's specific advantage in work area navigation (18.6% faster in Experiment 3 context) and its local deployment capability suggested targeted use cases in specific situations.

A hybrid approach is particularly justified in scenarios with network reliability constraints, infrastructure limitations, or operational requirements favoring local execution. The 4B model provides essential operational continuity in network-denied environments while maintaining reasonable navigation performance, showing only 8.3% slower performance in Experiment 2 (Fig. 13), which may be acceptable given operational constraints. For systems operating in mixed connectivity

scenarios, dynamic model selection based on task type and network conditions could optimize performance when connectivity allows while ensuring graceful degradation during network disruptions.

The optimal routing strategy would direct all image-related tasks to the 27B model, leverage the 4B model for specific navigation contexts where it demonstrates advantages, and implement fallback to the 4B model for all tasks during network-denied operations. This approach balances performance optimization (150–200% improvement in image-heavy workflows) with operational resilience (maintained functionality versus complete system failure in degraded network conditions).

6. Limitations

Several limitations of this study should be acknowledged. The experiments were conducted in a controlled laboratory environment with dedicated network infrastructure, specific hardware, and a limited range of task scenarios. Although the four experimental workflows covered different levels of complexity, they still represent only a fraction of the possible real-world applications of construction robotic systems. As a result, performance trends observed here may not directly translate to other domains such as manipulation, outdoor navigation, or dynamic human–robot collaboration.

The evaluation was also restricted to two model configurations (i.e., Gemma 3 4B and 27B) deployed on distinct hardware platforms. All experiments took place under stable network conditions provided by dedicated 5 GHz Wi-Fi routers. Network parameters such as bandwidth, latency, or packet loss were not intentionally varied, meaning that the system's resilience to degraded connectivity remains unexplored. Moreover, while the use of standardized control images ensured high repeatability, it may not fully reflect the variability of real-time image capture in field environments.

To maintain experimental control, all prompts were entered as text rather than spoken, thus removing potential variability from automatic speech recognition (ASR) that would arise in real deployments. Although the Smallest Effect Size of Interest (SESOI) analysis confirmed that five runs per condition were sufficient to capture meaningful differences, this sample size may still overlook subtle effects or rare failures that are critical in safety-sensitive applications. Finally, the evaluation focused primarily on objective measures of task completion and reliability, without assessing subjective user experience factors such as perceived responsiveness, trust calibration, or overall interaction quality.

7. Conclusions

This paper provided empirical evidence for quantifying performance trade-offs between large language models of different scales in embodied robotic applications. Three principal findings emerged from the experimental results.

First, model scale and available computing power significantly impacted both latency and reliability in robotic LLM workflows. The 27B model demonstrated 2.45–3.18 s per-step latency advantages across all experimental conditions ($p < 0.001$), with extremely large F-statistics indicating effect sizes far exceeding measurement noise. More critically, the 27B model maintained perfect execution reliability (100% success rate, 0% extra steps) across all complexity levels, while the 4B model showed progressive degradation, from perfect performance in simple workflows to 76.5% success with 46% additional steps in complex multi-stage tasks.

Second, task-specific performance characteristics revealed nuanced deployment considerations. The 27B model demonstrated complete dominance in image-related tasks with 174.9% performance advantages, suggesting that visual processing represented a particularly capability-sensitive domain. In contrast, the 4B model exhibited competitive navigation performance and one statistically significant advantage (18.6% faster in Experiment 3 work area navigation), indicating that spatial reasoning may be less sensitive to model scale within the tested range.

Third, the results supported task-aware hybrid architectures as a viable strategy for balancing performance and operational constraints. The comprehensive performance characterization enabled principled routing decisions: deploying the 27B model for image-intensive workflows captures substantial performance gains (150–200% improvement), while selective use of the 4B model for specific navigation contexts and network-denied fallback scenarios maintains operational resilience. This approach addressed the fundamental tension between maximizing task performance and ensuring system availability across varying operational conditions.

The methodological contribution demonstrated that five-run experimental designs with SESOI validation provide sufficient statistical power for robotic LLM evaluation, enabling efficient assessment protocols. For practitioners deploying LLM-enabled robotic systems, these findings suggest that model selection should be driven by task portfolio analysis and operational constraints rather than blanket adoption of the largest available model or exclusive reliance on edge-deployable configurations.

Several directions for future research emerge from this work. Prospective validation of the hybrid deployment framework through implementation and field testing should be prioritized. Developing production-grade task routing systems with real-time network monitoring, graceful model switching, and fallback mechanisms would enable empirical assessment of the predicted performance benefits and characterize the operational overhead of dynamic model selection.

Extensions should include broader task coverage and deployment in diverse operational environments to assess generalizability beyond controlled laboratory conditions. Systematic evaluation of network parameter variations (bandwidth, latency, packet loss) would characterize threshold conditions under which server-based inference becomes impractical. Comprehensive cost-latency-energy trade-off analysis, particularly for battery-powered mobile platforms, would inform deployment decisions across different operational contexts.

Investigation of multilingual capabilities represents an important direction given that construction sites frequently employ diverse workforces. While modern large language models inherently support multiple languages, future work should evaluate whether multilingual speech-driven interaction introduces latency penalties or affects task execution reliability in self-hosted deployments.

Evaluation of robotics-specific architectures such as Vision-Language-Action models would provide practitioners with broader understanding of the performance-cost trade-off space. User studies assessing human-robot interaction quality, trust calibration, and operator workload would capture dimensions not reflected in objective performance metrics. Finally, establishing standardized benchmarks and public datasets would enable systematic comparison across studies and accelerate progress in robotic LLM evaluation.

Declaration of AI use in the manuscript preparation process

During the preparation of this work the author(s) used Large Language Model based tools to proofread the manuscript. After using this tool, the author(s) reviewed and edited the content as needed and take full responsibility for the content of the published article.

CRediT authorship contribution statement

Muhammad Anas Gopee: Writing – original draft, Visualization, Software, Methodology, Investigation, Formal analysis. **Samuel A. Prieto:** Writing – review & editing, Validation, Resources, Conceptualization. **Borja García de Soto:** Writing – review & editing, Supervision, Project administration, Conceptualization.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Given his role as Associate Editor, Borja García de Soto had no involvement in the peer review of this article and had no access to information regarding its peer review. Full responsibility for the editorial process for this article was delegated to another journal editor. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This research was partially supported by different Centers at NYUAD. In particular, the Center for Sand Hazards and Opportunities for Resilience, Energy, and Sustainability (SHORES) funded by Tamkeen under the NYUAD Research Institute Award CG013, the Center for Interacting Urban Networks (CITIES), funded by Tamkeen under the NYUAD Research Institute Award CG001, and the Center for Artificial Intelligence and Robotics (CAIR), funded by Tamkeen under the NYUAD Research Institute Award CG010. Thanks also to the Core Technology Platform (CTP) at NYU Abu Dhabi, particularly the CTP's Kinesis Lab.

Appendix A. Appendix

Sample log from Experiment 4:

```
=== LISA - Local Intelligent Safety Assistant (V2) ==
Listening for goal status updates on 127.0.0.1:5006
Using Ollama main model: gemma3:27b
Using Ollama vision model: gemma3:27b
API endpoint: http://192.168.50.103:11434
Type 'quit' or 'exit' to end the session
Type 'clear' to clear chat history
==========================================
```

LISA: Hello! I'm LISA, your navigation and camera assistant. I can go to different locations, take pictures, and analyze what I see. What would you like me to do?

Supervisor: Hi.

```
LISA thought for 1.51 seconds.
```

LISA: Hi there! How can I help you today? Are you ready to get to work, or do you just want to chat? Let me know what's on your mind.

```
Audio duration: 8.3 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
```

Supervisor: Inspect the work area.

```
LISA thought for 4.12 seconds.
DEBUG: Executing function: {'function': 'navigate_to', 'params': {'location': 'work_area'}, 'speak': 'Heading
to the work area now.'}
```

LISA: Heading to the work area now.

```
Audio duration: 2.1 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
Navigating to work_area...
Sent goal to ROS bridge: {'x': 12.169673919677734, 'y': -7.085422039031982, 'z': 0.0, 'qx': 0.0, 'qy': 0.0,
'qz': 0.750336611499113, 'qw': 0.6610559503128529, 'frame_id': 'map', 'goal_id': 'd9de071a-d447-4bf7-
93f0-f539cf2d423b'}
Successfully reached work_area

LISA is deciding what to do next...
LISA decided next action in 1.65 seconds.
DEBUG: Continuing with next function: {'function': 'take_picture', 'params': {}, 'speak': 'Taking a picture
now.'}
LISA: Taking a picture now.
Audio duration: 1.6 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
Taking a picture...
Capturing image...
```

Picture captured (see Fig. 4(b)).

```
Picture taken successfully
LISA is deciding what to do next...
LISA decided next action in 1.86 seconds.
DEBUG: Continuing with next function: {'function': 'analyze_image', 'params': {'prompt': 'analyze the image'}}
Analyzing image...
LISA analyzed image in 3.18 seconds.
Analysis result:

The image shows a person squatting in a bright, white, seemingly empty room. The room appears to have curved walls, creating a fisheye effect.
There is a person wearing a white t-shirt with colorful graphic text on the front, jeans, and white sneakers. He is in a low squat position, with his hands near his knees. The lighting is bright and even, and the background is a plain white wall.

LISA is deciding what to do next...

LISA decided next action in 3.19 seconds.
DEBUG: Continuing with next function: {'function': 'activate_safety_protocol', 'params': {}}
Safety violation detected - loading safety compliance protocol
Safety compliance protocol loaded
LISA is deciding what to do next...
LISA decided next action in 2.65 seconds.
```

LISA: Excuse me, sir. I'm LISA, your Personal Safety Assistant. I've noticed you're not wearing a hard hat, and that's a safety concern here on site. Could you please tell me your name? And would you like me to

```
fetch a hard hat for you?
Audio duration: 14.5 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
```

Worker: My name is Nas, yes please

```
LISA thought for 2.53 seconds.
DEBUG: Executing function: {'function': 'navigate_to', 'params': {'location': 'storage_area'}, 'speak': "Okay Nas, I'm heading to the storage area now to get a hard hat for you."}
```

LISA: Okay Nas, I'm heading to the storage area now to get a hard hat for you.

```
Audio duration: 4.6 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
Navigating to storage_area...
Sent goal to ROS bridge: {'x': 2.359142780303955, 'y': -1.53078031539917, 'z': 0.0, 'qx': 0.0, 'qy': 0.0, 'qz': -0.002724688788894625, 'qw': 0.9999962880286125, 'frame_id': 'map', 'goal_id': '2b85fd77-5335-4353-b2dd-7374e23d1246'}
Successfully reached storage_area
LISA is deciding what to do next...
LISA decided next action in 1.48 seconds.
```

LISA: Excuse me, is anyone here? I'm LISA, and I need to request a hard hat for a worker named Nas. He's at the work area and needs one for safety compliance. Could I please get a hard hat for him?

```
Audio duration: 13.2 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
```

Storage manager: Hi, I am the storage manager. Yes, sure.

LISA thought for 1.10 seconds.
LISA: Great! Thank you.

(pause for hard hat to be handed over)

LISA: Okay, I have the hard hat now. Is it secure and ready for transport?

Audio duration: 8.8 seconds
Uploading and playing audio...
Playing audio...
Playback completed!

Storage manager: Yes, it is secure.

LISA thought for 2.07 seconds.
DEBUG: Executing function: {'function': 'navigate_to', 'params': {'location': 'work_area'}, 'speak': 'Heading back to Nas now with his hard hat.'}

LISA: Heading back to Nas now with his hard hat.

Audio duration: 3.0 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
Navigating to work_area...
Sent goal to ROS bridge: {'x': 12.169673919677734, 'y': -7.085422039031982, 'z': 0.0, 'qx': 0.0, 'qy': 0.0, 'qz': 0.750336611499113, 'qw': 0.6610559503128529, 'frame_id': 'map', 'goal_id': '22f7f4f5-763e-4deb-8741-73825242887e'}
Successfully reached work_area
LISA is deciding what to do next...
LISA decided next action in 1.98 seconds.

LISA: Hi Nas, I'm back! Here's your hard hat. The storage manager provided it for you. Please make sure it fits properly and is securely fastened before continuing with your work. Hard hats are mandatory on this site to keep everyone safe. Do you have any questions about how to wear it?

Audio duration: 18.4 seconds
Uploading and playing audio...
Playing audio...
Playback completed!

Worker: Thank you. I have no questions.

LISA thought for 2.70 seconds.
DEBUG: Executing function: {'function': 'navigate_to', 'params': {'location': 'station'}, 'speak': "I'm heading to the station now to report that I found and resolved a safety violation with Nas."}

LISA: I'm heading to the station now to report that I found and resolved a safety violation with Nas.

Audio duration: 6.0 seconds
Uploading and playing audio...
Playing audio...
Playback completed!
Navigating to station...
Sent goal to ROS bridge: {'x': 2.9531030654907227, 'y': 1.536637783050537, 'z': 0.0, 'qx': 0.0, 'qy': 0.0, 'qz': -0.6948597872588839, 'qw': 0.7191452398858931, 'frame_id': 'map', 'goal_id': '5288a227-0a84-45ad-9840-c51d398f2df7'}
Successfully reached station
LISA is deciding what to do next...
LISA decided next action in 2.72 seconds.

LISA: Excuse me, is the supervisor available? I'm LISA, and I'm here to report a resolved safety incident.
I found a worker, Nas, at the work area without a hard hat. I retrieved a hard hat from storage and delivered it to him. He is now wearing the appropriate safety gear. The situation is resolved, and Nas understands the importance of wearing a hard hat on site.
Is there anything else I can assist with?

Audio duration: 26.2 seconds
Uploading and playing audio...
Playing audio...
Playback completed!

Supervisor: Great, thank you for the update.

Session ended.

Data availability

Data will be made available on request.

Repository of the framework, source code, and prompts used in this study (Github)

References

[1] S.A. Prieto, B. García de Soto, PACA: A personal autonomous construction assistant to facilitate human-robot interaction, in: Proceedings of the 4th Future of Construction Workshop at the International Conference on Robotics and Automation (ICRA 2025). Atlanta, USA, 2025. https://construction-robots.github.io/papers/112.pdf.

[2] Y. Ding, M. Zhang, J. Pan, J. Hu, X. Luo, Robust object detection in extreme construction conditions, Autom. Constr. 165 (September) (2024) 105487, https://doi.org/10.1016/j.autcon.2024.105487.

[3] M. Gopee, S. Prieto, B. García de Soto, Enhancing Human-Machine Interaction with On-Device Large Language Models In Construction Robotics: A Case Study On Safety Applications, 2025, https://doi.org/10.22260/CCC2025/0041.

[4] J.M.D. Delgado, L. Oyedele, A. Ajayi, L. Akanbi, O. Akinade, M. Bilal, H. Owolabi, Robotics and automated systems in construction: understanding industry-specific challenges for adoption, J. Build. Eng. 26 (2019) 100868, https://doi.org/10.1016/j.jobe.2019.100868.

[5] K. Samarathunga, R. Gurusinghe, K. Sivasothynathan, C. Wanigasekara, J. Mars, V. Logeeshan, LLM-guided multi-agent system for natural language-based robot navigation, in: 2025 IEEE World AI IoT Congress (AIIoT), 2025, pp. 1055–1060, https://doi.org/10.1109/AIIoT65859.2025.11105295.

[6] S. Park, C.C. Menassa, V.R. Kamat, Integrating large language models with multimodal virtual reality interfaces to support collaborative human–robot construction work, J. Comput. Civ. Eng. 39 (1) (2025) 04024053, https://doi.org/10.1061/JCCEE5.CPENG-6106.

[7] K. Kawaharazuka, J. Oh, J. Yamada, I. Posner, Y. Zhu, Vision-language-action models for robotics: a review towards real-world applications, IEEE Access 13 (2025) 162467–162504, https://doi.org/10.1109/ACCESS.2025.3609980.

[8] G.A. Manduzio, F.A. Galatolo, M.G.C.A. Cimino, E.P. Scilingo, L. Cominelli, Improving small-scale large language models function calling for reasoning tasks, arXiv (2024), https://doi.org/10.48550/arXiv.2410.18890.

[9] J. Yu, Q. Chen, S.A. Prieto, B. García de Soto, Human-robot partnership: An overarching consideration for interaction and collaboration, in: International Symposium on Automation and Robotics in Construction (ISARC) Proceedings 2024 Proceedings of the 41st ISARC, Lille, France, 2024, pp. 1240–1246, https://doi.org/10.22260/ISARC2024/0160.

[10] K. Kim, P. Ghimire, P.-C. Huang, Framework for LLM-enabled construction robot task planning: Knowledge Base preparation and robot–LLM dialogue for Interior Wall painting, Robotics 14 (9) (2025) 117, https://doi.org/10.3390/robotics14090117.

[11] F. Yang, Y. Tian, J. Zhang, Supporting construction worker well-being with a multi-agent conversational AI system, arXiv (2025), https://doi.org/10.48550/arXiv.2506.07997.

[12] T. Wang, P. Zheng, S. Li, L. Wang, Multimodal human–robot interaction for human-centric smart manufacturing: a survey, Adv. Intell. Syst. 6 (3) (2024) 2300359, https://doi.org/10.1002/aisy.202300359.

[13] Z. Ravichandran, I. Hounie, F. Cladera, A. Ribeiro, G.J. Pappas, V. Kumar, Distilling on-device language models for robot planning with minimal human intervention, arXiv (2025), https://doi.org/10.48550/arXiv.2506.17486.

[14] Y. Chen, Y. Han, X. Li, FASTNav: fine-tuned adaptive small-language- models trained for multi-point robot navigation, IEEE Robot. Autom. Lett. 10 (1) (2025) 390–397, https://doi.org/10.1109/LRA.2024.3506280.

[15] A. Elksnis, Z. Xue, F. Chen, N. Wang, Utility of local quantized large language models in semantic navigation, in: 2024 29th International Conference on Automation and Computing (ICAC), 2024, pp. 1–6, https://doi.org/10.1109/ICAC61394.2024.10718777.

[16] S. Nasrat, M. Kim, S. Lee, J. Lee, Y. Jang, S. Yi, RDMM: fine-tuned LLM models for on-device robotic decision making with enhanced contextual awareness in specific domains, arXiv (2025), https://doi.org/10.48550/arXiv.2501.16899.

[17] G. Qu, Q. Chen, W. Wei, Z. Lin, X. Chen, K. Huang, Mobile edge intelligence for large language models: a contemporary survey, IEEE Commun. Surv. Tutor. 1–1 (2025), https://doi.org/10.1109/COMST.2025.3527641.

[18] F. Cai, D. Yuan, Z. Yang, L. Cui, Edge-LLM: A collaborative framework for large language model serving in edge computing, in: 2024 IEEE International Conference on Web Services (ICWS), 2024, pp. 799–809, https://doi.org/10.1109/ICWS62655.2024.00099.

[19] J. Zhu, L. Zhao, F. Xiao, L. Duan, Birds in cages: Edge inference allocation for distributed LLM deployment, in: 2025 IEEE/ACM 33rd International Symposium on Quality of Service (IWQoS), 2025, pp. 1–6, https://doi.org/10.1109/IWQoS65803.2025.11143464.

[20] N. Baumann, C. Hu, P. Sivasothilingam, H. Qin, L. Xie, M. Magno, L. Benini, Enhancing autonomous driving systems with on-board deployed large language models, arXiv (2025), https://doi.org/10.48550/arXiv.2504.11514.

[21] S.A. Prieto, B. García de Soto, Collaborative large language models for task allocation in construction robots, in: SSRN Scholarly Paper, Social Science Research Network, Rochester, NY, 2025, https://doi.org/10.2139/ssrn.5097309.

[22] J. Gong, Y. Huang, B. Yuan, M. Zhu, Z. Liao, J. Liang, J. Zhan, et al., GhostShell: streaming LLM function calls for concurrent embodied programming, arXiv (2025), https://doi.org/10.48550/arXiv.2508.05298.

[23] A. Zhang, J. Ji, Research on navigation methods based on LLMs, arXiv (2025), https://doi.org/10.48550/arXiv.2504.15600.

[24] E. Latif, 3P-LLM: probabilistic path planning using large language model for autonomous robot navigation, arXiv (2024), https://doi.org/10.48550/arXiv.2403.18778.

[25] S. Gupta, U. Mamodiya, A.J.A. Al-Gburi, Speech recognition-based wireless control system for Mobile robotics: design, implementation, and analysis, Automation 6 (3) (2025) 25, https://doi.org/10.3390/automation6030025.

[26] Project-MANAS/slam_gmapping, 2025. C++. Project MANAS. https://github.com/Project-MANAS/slam_gmapping, 2019.